

A Semantic Autonomy Framework for VLM-Integrated Indoor Mobile Robots: Hybrid Deterministic Reasoning and Cross-Robot Adaptive Memory

Authors: Bogdan Felician Abaza, Andrei-Alexandru Staicu and Cristian Vasile Doicin

Affiliations: Faculty of Industrial Engineering and Robotics (FIIR), National University of Science and Technology POLITEHNICA Bucharest

Corresponding author: Bogdan Felician Abaza, bogdan.abaza@upb.ro

Highlights

- Hybrid resolver bypasses VLM in 88% of navigation decisions
- Cross-robot memory transfer succeeds in 33/33 tested missions
- Learned preferences reduce repeated decisions to sub-millisecond latency
- Semantic memory supports cross-session and cross-robot adaptation
- Validated on physical ROS 2 robots using off-the-shelf edge hardware, no GPU

Abstract

Autonomous indoor mobile robots can navigate reliably to metric coordinates using established frameworks such as ROS 2 Navigation 2, yet they lack the ability to interpret natural language instructions that express intent rather than positions. Vision-Language Models offer the semantic reasoning required to bridge this gap, but their inference latency (2-9 seconds per decision on consumer hardware) and session-by-session amnesia limit practical deployment.

This paper presents the Semantic Autonomy Stack, a six-layer reference framework for semantically autonomous indoor navigation, and validates a complete instance featuring hybrid deterministic-VLM reasoning and cross-robot adaptive memory on physical robots with off-the-shelf edge hardware. A seven-step parametric resolver handles 88% of instructions in under 0.1 milliseconds without invoking a language model, camera, or GPU; only genuinely ambiguous instructions escalate to VLM reasoning. A five-category semantic memory framework with explicit scope taxonomy (global environment knowledge, per-operator preferences, per-robot capabilities) enables cross-session learning and cross-robot knowledge transfer: preferences learned through VLM interactions on one robot are promoted to deterministic resolution and transferred to a second robot via a shared compiled digest, achieving a measured latency reduction of 103,000-fold.

Experimental validation on two custom-built differential-drive robots across 82 scenario-level decisions and three sessions demonstrates 100% semantic transfer accuracy (33/33, 95% CI [0.894, 1.000]), 100% semantic resolution accuracy, and concurrent multi-robot operation feasibility - all on Raspberry Pi 5 platforms with no onboard GPU, requiring zero training data.

Keywords: semantic navigation; vision-language model; dual-process reasoning; cross-robot transfer; ROS 2; edge robotics

1. Introduction

Autonomous indoor navigation for mobile robots has reached a high level of maturity with the widespread adoption of the Robot Operating System 2 (ROS 2) and its Navigation 2 (Nav2) stack [1]. Contemporary platforms routinely achieve reliable point-to-point navigation using occupancy grid maps, adaptive Monte Carlo localization (AMCL), and model-predictive controllers such as MPPI [2]. However, these systems operate on metric coordinates: the robot navigates to a pose specified in the map frame without understanding why it should go there or what it will find upon arrival. In computer-integrated manufacturing, intralogistics, and shared human-robot workspaces, this limitation becomes critical when mobile robots are expected to respond to task-level requests, interact with operators, and adapt to semantically meaningful locations such as workstations, storage areas, safety equipment, rest zones, restricted-access rooms, or temporary operational zones. Recent RCIM research has identified large language models as a promising enabling technology for new-generation intelligent manufacturing, particularly for semantic interpretation, planning support, and human-machine interaction [3].

This coordinate-centric paradigm becomes insufficient when robots must respond to natural language instructions that express intent rather than coordinates. An operator who says “take me somewhere I can sit and relax” expects the robot to identify an appropriate location based on the semantic properties of the environment - furniture

attributes, spatial context, and prior experience - rather than receiving explicit coordinates. Vision-Language Models (VLMs) have emerged as a promising mechanism for bridging this semantic gap, enabling robots to interpret instructions, reason about object affordances, and select navigation targets grounded in visual and spatial understanding [4].

Despite this promise, three practical challenges still limit the deployment of VLM-based semantic navigation on physical robots operating in real indoor environments.

Challenge 1: Inference latency. VLM inference on consumer-grade hardware requires 2–9 seconds per decision depending on model size, prompt complexity, and GPU capability. For routine instructions that occur frequently and have unambiguous resolutions - such as “go to the lab” or “take me to the restroom” - this latency is unnecessary and impractical for continuous operation. A navigation system that invokes a multi-billion parameter model for every instruction, regardless of complexity, imposes a computational and temporal cost disproportionate to the reasoning required.

Challenge 2: Session amnesia. Current VLM-based navigation systems treat each operating session independently. Knowledge acquired during operation - which instructions the operator uses, which locations they prefer, which routes succeed reliably - is discarded at shutdown. When the robot restarts, it must re-learn the same patterns through repeated VLM interactions. To the best of our knowledge, no published physical multi-robot navigation system has demonstrated persistent semantic memory that promotes VLM-resolved instructions into deterministic resolution rules across sessions and transfers these learned preferences across robots operating in the same environment.

Challenge 3: Limited physical multi-robot validation and cross-robot transfer. Experimental evaluation of VLM-based navigation is still frequently conducted in simulation environments, where perception can be idealized and localization errors are either absent or tightly controlled [4]. When physical validation is reported, it commonly focuses on a single robot operating in a controlled environment. More, learned semantic associations are rarely evaluated for transfer across physical robots sharing the same map, semantic annotations, and operator context. This limits confidence in the scalability of VLM-based navigation architectures toward practical multi-robot deployments.

1.1. Dual-process reasoning for robot navigation

Cognitive science’s dual-process theory distinguishes between fast, automatic responses (System 1) and slow, deliberative reasoning (System 2) [5]. This distinction has been applied to VLM-based robot navigation: recent work proposed training a classifier to route instructions to either a fast or a slow processing path, achieving a 53.6% fast-path rate on indoor navigation tasks [6].

The approach presented in this paper pursues the same dual-process objective through a fundamentally different mechanism. Rather than training a classifier - which requires labeled training data, GPU computation, and periodic retraining as the environment changes - the system employs a seven-step parametric cascade that exploits the structure of the navigation graph, the attributes of semantically annotated points of interest, and patterns learned from prior VLM interactions. This cascade requires no training data, no GPU, and no model updates. It operates in under 0.1 milliseconds on off-the-shelf ARM hardware, resolving 88% of instructions without invoking the VLM or acquiring a camera image. When the cascade cannot produce an unambiguous resolution, the instruction escalates to VLM reasoning - preserving the deliberative capability for genuinely ambiguous cases.

1.2. Contributions

This paper makes four contributions:

C1 - Semantic Autonomy Stack (SAS). A six-layer reference framework (L0–L5) defining the capabilities required for semantically autonomous indoor navigation. The framework separates platform hardware (L0), navigation execution (L1), perception (L2), semantic reasoning (L3), mission interface (L4), and operational intelligence with memory (L5). Each layer prescribes capabilities without mandating specific implementations, enabling systematic component replacement and cross-platform portability. Unlike a purely conceptual taxonomy, the proposed SAS is instantiated and experimentally validated through a working ROS 2/Nav2 implementation on two physical mobile robots.

C2 - Hybrid deterministic-VLM reasoning. A seven-step parametric resolver (L3a) that bypasses VLM inference for the majority of instructions in under 0.1 ms, with automatic escalation to VLM reasoning (L3b) only when deterministic resolution is ambiguous. The fast path requires no training data, no model updates, and no GPU acceleration. The resolver incorporates a preference-based step (Step 0) that converts previously learned VLM-mediated decisions into deterministic resolution rules, creating a measurable adaptation mechanism. The complete learning cycle - from initial VLM reasoning through memory promotion to deterministic cross-robot transfer - is demonstrated experimentally, with a measured latency reduction exceeding 103,000×.

C3 - Cross-robot adaptive memory. A five-category semantic memory framework (M1–M5) with explicit scope taxonomy: environment knowledge (global, per-map), temporal patterns (global), operator preferences (global,

per-operator), platform capabilities (per-robot), and task history (per-robot, aggregable). Operator preferences learned through VLM interactions on one robot are promoted to deterministic resolution rules and made available to a second robot through a shared compiled memory digest, achieving 100% semantic transfer accuracy across 33 experimental decisions without retraining or per-robot rule engineering.

C4 - Physical multi-robot validation. Experimental validation on two custom-built differential-drive robots across 82 scenario-level decisions and three sessions, using Raspberry Pi 5 platforms for onboard navigation and a shared consumer GPU workstation for VLM inference. The validation demonstrates cross-robot memory transfer, the complete learning cycle from VLM reasoning to deterministic reuse, deterministic consistency across platforms, and concurrent operation feasibility under real sensing noise and localization uncertainty.

1.3. Relationship to prior work

This paper extends the authors’ prior work on lightweight semantic-aware route planning with monocular camera–2D LiDAR fusion [7], which established the lower SAS layers L1–L2: Nav2-based navigation execution, Angular Sector Fusion (ASF) for object localization in the map frame, persistent semantic mapping with mobility-based time-to-live and exponential moving average position fusion, and penalty-weighted route selection through the Nav2 Route Server. That work validated perception-to-routing integration on three physical robots across 115 navigation legs with a 97% overall success rate.

The present paper builds upon that validated L1–L2 foundation by introducing and experimentally validating the higher SAS layers: L3, through hybrid deterministic–VLM semantic reasoning, and L5, through adaptive semantic memory with cross-session learning and cross-robot transfer. Thus, the contribution of the present work is not a repetition of perception-to-routing validation, but the extension of the stack from semantic perception toward intent resolution, memory-driven adaptation, and multi-robot knowledge transfer.

1.4. Paper organization

The remainder of this paper is organized as follows. Section 2 reviews related work in VLM-based navigation, dual-process reasoning for robotics, semantic memory, and edge deployment. Section 3 introduces the SAS reference framework and maps its layers to the validated implementation. Section 4 describes the hybrid L3a/L3b reasoning architecture, including the seven-step deterministic resolver, VLM integration, visual confirmation, and executive contract. Section 5 presents the five-category semantic memory framework with cross-robot transfer mechanism. Section 6 details the experimental methodology. Section 7 reports the results. Section 8 discusses the findings, compares with related work, and identifies limitations. Section 9 concludes and outlines future work.

2. Related Work

This section reviews four areas of related research that define the positioning of the proposed framework: VLM-based robot navigation, dual-process reasoning for robotics, semantic memory and cross-session learning, and LLM–ROS integration frameworks. For each area, we identify the specific gap addressed by the present work: selective VLM invocation, deterministic reuse of learned semantic preferences, and physical cross-robot transfer in a ROS 2/Nav2 deployment.

2.1. VLM-based robot navigation

The integration of Vision-Language Models into mobile robot navigation has advanced rapidly. Recent surveys [4, 8] classify approaches along two axes: how the VLM is integrated into the control loop (end-to-end vs. modular) and where the system is validated (simulation vs. physical hardware).

End-to-end VLA models. Vision-Language-Action (VLA) models unify visual perception, language understanding, and action generation within a single sequence modeling framework [9, 10]. VLM2VLA demonstrated that representing robot actions as natural language preserves VLM reasoning capabilities while enabling manipulation control, retaining over 85% of VQA performance after fine-tuning [10]. MemoryVLA introduced a perceptual-cognitive memory bank for long-horizon manipulation tasks, addressing session amnesia at the action level [11]. While these architectures advance the frontier of end-to-end robot learning, they target manipulation rather than navigation and require substantial training data. The present work addresses semantic target resolution for navigation without task-specific training, while relying on pretrained perception and vision-language models only when required.

Modular VLM navigation. Modular approaches preserve the conventional perception-planning-control pipeline while using VLMs for specific reasoning tasks. ReasonNav demonstrated human-like navigation skills (reading signs, asking for directions) by leveraging VLM reasoning on landmark abstractions in large buildings [12]. VLM-Social-Nav used a VLM-based scoring module to generate social compliance costs for motion planning, achieving a 19% reduction in collision rates across four real-world indoor scenarios [13]. Wang et al. proposed VLM-based human-guided navigation for smart manufacturing environments, combining 3D scene reconstruction with zero-shot semantic segmentation [14]. These modular systems demonstrate that VLMs can

enhance specific navigation capabilities, including landmark-based reasoning, socially compliant motion, and human-guided navigation in manufacturing environments. However, their semantic reasoning stage remains tightly coupled to VLM inference for the targeted high-level decisions. The present work introduces a selective invocation mechanism in which routine or previously learned instructions are resolved deterministically before any VLM call is made.

Co-NavGPT extends modular VLM-based navigation to a multi-robot setting by using a VLM as a global planner for cooperative visual semantic navigation [15]. The framework aggregates observations from multiple robots into a global semantic map and assigns exploration frontiers based on spatial context and target-object semantics. While Co-NavGPT demonstrates the value of VLMs for multi-robot coordination, its focus is cooperative target search in initially unknown environments. The present work addresses a complementary problem: transferring learned operator preferences across physical ROS 2/Nav2 robots operating on a shared semantic map, so that instructions initially resolved through VLM reasoning can later be handled deterministically without invoking the model.

Simulation-dominant validation. NaviTrace [16] highlighted a persistent challenge: evaluating VLM navigation capabilities remains constrained by costly real-world trials and limited benchmarks. The benchmark evaluated eight state-of-the-art VLMs on 2D trace prediction, finding that goal localization rather than path planning was the dominant failure mode. RANa [17] introduced retrieval-augmented navigation where agents query a database of observations from previous episodes, demonstrating cross-episode knowledge transfer in simulation. The present work addresses the complementary physical-deployment problem: whether semantic associations learned during VLM-mediated navigation can be promoted to deterministic rules and transferred across physical robots sharing the same map and semantic annotations.

2.2. Dual-process reasoning for robotics

Kahneman’s dual-process theory [5] - distinguishing fast, automatic System 1 responses from slow, deliberative System 2 reasoning - has been adopted by several robotics architectures.

IROS. The most directly comparable work is the IROS framework for real-time VLM-based indoor navigation, which separates fast reflexive decisions from slower VLM-based reasoning and reports a 53.6% System 1 routing rate with a 66% latency reduction across five real-world buildings [6]. The key architectural difference is the routing mechanism: IROS learns when to use the fast pathway, whereas the present work resolves routine instructions through a seven-step parametric cascade based on graph structure, semantic annotations, and promoted memory preferences. Because the evaluation protocols differ, the reported 88% fast-path rate in this work should be interpreted as evidence that the proposed deterministic cascade is effective in the tested semantic-navigation setting, rather than as a direct benchmark-level comparison.

CogDDN. Unlike CogDDN [18], which focuses on dynamically adjusting reasoning depth based on scene complexity, the present work focuses on persistent memory-driven adaptation: M3 preference matching allows instructions that previously required VLM reasoning to be resolved deterministically in later sessions.

Hydra-Nav. Wang et al. unified dual-process reasoning within a single VLM by adaptively switching between direct low-level execution and Chain-of-Thought reasoning for object navigation [19]. Their approach uses reinforcement learning to learn when to think deeply vs. act quickly. The present work differs in that the fast path is entirely external to the VLM: a parametric cascade that does not invoke any language model, does not acquire a camera image, and does not require GPU acceleration for routine instructions. In addition, while Hydra-Nav is evaluated on object-navigation benchmarks, the present work focuses on physical ROS 2/Nav2 deployment and cross-robot reuse of learned semantic preferences.

Dual Process VLA [20]. The navigation-specific contribution of the present work—a seven-step resolver that promotes previously VLM-mediated semantic decisions into deterministic target resolution—is orthogonal to this manipulation-oriented line of work.

2.3. Semantic memory and cross-session learning

Persistent memory for robot navigation remains an active research area with distinct approaches ranging from visual maps to causal graphs.

MapNav introduced Annotated Semantic Maps (ASM) as a structured memory representation for VLM-based vision-and-language navigation, replacing raw historical frame sequences with top-down semantic maps enriched by explicit textual labels [21]. This supports the broader trend toward structured memory representations in VLN. The present work differs by using memory not primarily as an input representation for continuous VLM inference, but as an operational mechanism for promoting previously VLM-mediated semantic decisions into deterministic rules that can be reused across sessions and transferred across robots.

CausalNav. Duan et al. proposed a dynamic causal semantic graph that integrates open-vocabulary understanding with real-time graph updates for outdoor navigation [22]. The system extends the RAG paradigm to dynamic environments through bidirectional LLM–RAG reasoning. CausalNav focuses on maintaining and updating a dynamic causal semantic graph for navigation. In contrast, the present work separates operational

memory into five categories (M1–M5) with explicit scope taxonomy-environment knowledge, temporal patterns, operator preferences, platform capabilities, and task history-so that shared knowledge and robot-specific knowledge can be treated differently during cross-robot transfer.

Meta-Memory. Meta-Memory addresses the question “where did I see object X?” through structured semantic-spatial retrieval [23]. The present work addresses a different operational question: “how should the robot resolve instruction X after it has observed consistent operator preferences?” This shifts the role of memory from spatial recall to deterministic reuse of previously VLM-mediated semantic decisions.

EchoVLA. This work introduced synergistic declarative memory (scene memory + episodic memory) for long-horizon mobile manipulation VLA models [24]. While architecturally sophisticated, EchoVLA integrates memory into the VLA inference process for long-horizon mobile manipulation. By contrast, the present work compiles selected memory content into a compact digest loaded at startup; promoted M3 preferences are then queried by a lightweight Jaccard matcher in under 0.1 ms, decoupling routine memory reuse from VLM inference.

Agent memory surveys. The rapid expansion of memory mechanisms for AI agents is documented in the “Memory in the Age of AI Agents” survey [25], which proposes a taxonomy of memory forms, functions, and dynamics. The present work’s M1–M5 framework maps onto this taxonomy: M1 (environment) and M2 (temporal) correspond to semantic memory, M3 (preferences) to procedural memory with promotion, M4 (platform) to embodiment-specific memory, and M5 (task history) to episodic memory with anomaly tracking.

2.4. LLM-ROS integration frameworks

The interface between language models and robot middleware has been formalized by several recent frameworks.

ROSClaw. Cardenas et al. presented a model-agnostic executive layer integrating the OpenClaw agent runtime with ROS 2, formalizing the interface as a contract: affordance manifest, observation normalizer, action validator, and audit logger [26]. Deployed on three platforms (wheeled, quadruped, humanoid) with four LLM backends, ROSClaw demonstrated that model choice significantly affects safety behavior (up to 4.8× differences in out-of-policy action rates). The present work adopts the ⟨A, O, V, L⟩ contract formalization and specializes it for semantic navigation: the affordance manifest includes graph nodes and POI attributes, the validator enforces navigation-specific safety checks, and the logger captures per-decision metrics with platform attribution for cross-robot analysis.

ROS-LLM. ROS-LLM operates at the level of translating natural-language instructions into executable ROS behaviors and supports feedback-driven embodied AI [27]. The present work is complementary: it focuses specifically on semantic navigation, where the central problem is resolving operator intent to a graph target, validating the action, executing Nav2 navigation, and confirming arrival.

ROSA and RAI. ROSA [28] provides a natural language overlay for ROS introspection and control. RAI [29] offers a vendor-agnostic agentic framework for physical AI robotics. Both provide general-purpose LLM-ROS interfaces without navigation-specific reasoning, memory, or cross-robot transfer. The present work addresses these gaps with a domain-specialized architecture.

Agentic and GLM-enhanced robotic systems. Raptis et al. surveyed 30 LLM-based robotic systems validated in real-world settings, introducing an “agenticness” classification based on autonomy, goal-directedness, adaptability, and decision-making [30]. Complementarily, recent RCIM reviews emphasize that multimodal language models and spatial intelligence are becoming key enablers for natural human–robot collaboration and human-centric smart manufacturing [31]. Ding et al. further frame GLM-enhanced HRC in Industry 5.0 as a perception–cognition–execution problem, highlighting the need to align multimodal perception, mutual cognition, and embodied execution [32]. The present work contributes to this direction in a narrower but experimentally grounded setting by demonstrating persistent cross-session learning and cross-robot transfer in a physical ROS 2/Nav2 semantic-navigation deployment.

3. The Semantic Autonomy Stack

This section introduces the Semantic Autonomy Stack (SAS), a six-layer reference framework that defines the capabilities required for semantically autonomous indoor mobile robot navigation. The framework is deliberately agnostic to specific sensors, models, and platforms: each layer prescribes what must be provided, not how it is implemented. We first define the SAS layers and their responsibilities, then compare the framework with existing architectural approaches and finally map the proposed structure to the physical Xplorer multi-robot implementation validated in this work.

3.1. Framework definition

The SAS organizes the capabilities of a semantically autonomous mobile robot into six layers (Fig. 1), ordered from the physical platform (L0) to operational intelligence (L5). The layered structure enforces separation of concerns: replacing a component within one layer (e.g., swapping a 2D LiDAR for a 3D depth camera at L2, or substituting one VLM for another at L3) requires no changes to adjacent layers.

L0 - Mobile platform. Locomotion hardware, onboard sensors, power, and compute. The platform may be differential-drive, omnidirectional, Ackermann, quadruped, or humanoid. L0 defines the physical envelope within which all other layers operate.

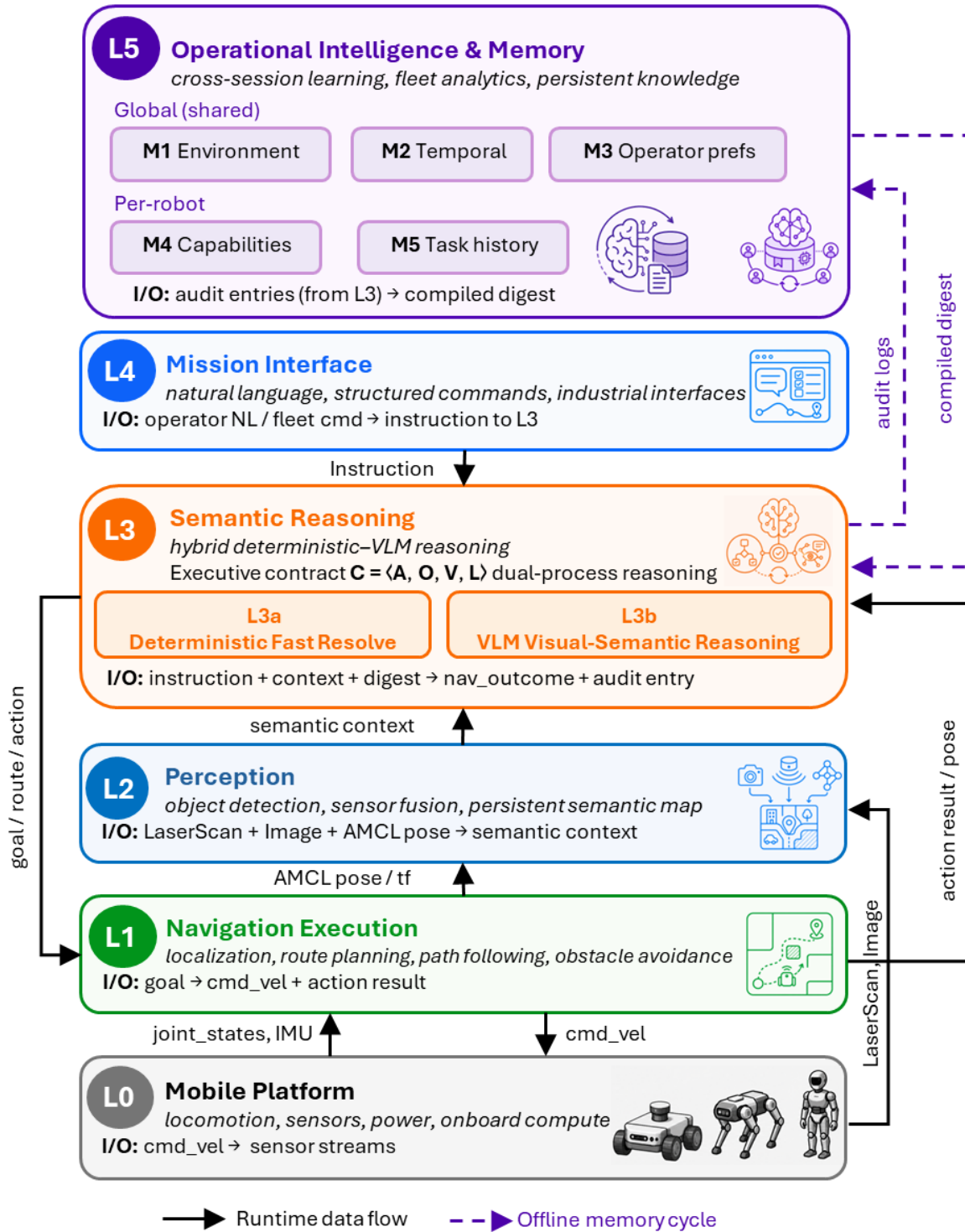


Fig. 1. The Semantic Autonomy Stack (SAS): six-layer reference framework and validated Xplorer multi-robot deployment. It summarizes the platform-agnostic SAS layers and their I/O interfaces.

L1 - Navigation execution. Localization, route planning, path following, and local obstacle avoidance. L1 receives a target pose or a route specification from L3 and executes it using the platform’s motion capabilities. Standard implementations include the ROS 2 Navigation 2 (Nav2) stack [1], proprietary AMR controllers, or hybrid configurations. L1 operates on metric coordinates and occupancy grids; it has no understanding of the semantic significance of the environment.

L2 - Perception. Object detection, sensor fusion, and persistent semantic map construction. L2 transforms raw sensor streams (camera images, LiDAR scans, depth maps) into localized semantic entities in the map frame. The output is a structured representation - such as a GeoJSON feature collection or a 3D scene graph - that L3 can query for reasoning. L2 may also propagate semantic annotations to L1 (e.g., as edge penalties in a navigation graph) to influence route selection without involving the reasoning layer. Our prior work [7] validated a complete L2 instance comprising Angular Sector Fusion (ASF) for monocular camera–2D LiDAR object localization, a persistent semantic map with mobility-based TTL and EMA position fusion, and penalty-weighted annotation of Nav2 Route Server graph edges.

L3 - Semantic reasoning. The layer that bridges the semantic gap between human intent and robot action. L3 receives a natural language instruction (or a structured mission specification) and resolves it to a concrete navigation target within the graph defined by L1–L2. The SAS prescribes a layered internal structure for L3:

- **L3a - Deterministic fast resolve.** A parametric cascade that attempts to resolve the instruction using graph structure, object identifiers, attributes, and learned preferences, without invoking a language model or acquiring a camera image. L3a operates in sub-millisecond time and requires no GPU acceleration for the deterministic fast path.
- **L3b - VLM visual-semantic reasoning.** Activated only when L3a cannot produce an unambiguous resolution. L3b acquires the current camera image, constructs a prompt incorporating the semantic context (graph, objects, memory), and queries a Vision-Language Model. In the validated implementation, L3b latency is typically 2–9 s on the shared consumer GPU workstation, depending on prompt length, visual context, and model response time.

This layered structure within L3 is the architectural expression of dual-process reasoning [5] applied to robot navigation: the fast, low-cost deterministic path handles routine instructions, while the slow, resource-intensive VLM path is reserved for genuinely ambiguous cases. L3 also includes post-arrival visual confirmation (VLM verification that the expected objects are visible at the destination) and an executive contract $C = \langle A, O, V, L \rangle$ that formalizes affordance injection, observation normalization, action validation, and audit logging, adopted from the ROSClaw framework [26].

L4 - Mission interface. The boundary between external systems and the robot’s reasoning layer. L4 accepts instructions in natural language, structured commands, or industrial protocol messages (e.g., VDA 5050 [33]) and translates them into the internal representation consumed by L3. The interface is extensible to additional protocols (MQTT, REST) and operates independently of both the robot platform and the reasoning workstation - instructions may originate from any ROS 2 DDS client, a web dashboard, or an industrial fleet manager. In our implementation, L4 is realized through a ROS 2 topic (*/vlm_instruction*) and an OpenClaw terminal user interface.

L5 - Operational intelligence and semantic memory. Cross-session learning, fleet analytics, and persistent knowledge management. L5 encompasses the five-category semantic memory framework (M1–M5, detailed in Section 5) that stores environment knowledge, temporal patterns, operator preferences, platform capabilities, and distilled task history. L5 also includes operator-facing tools for retrospective analysis (OpenClaw agent querying audit logs) and VLM interaction logging (per-mission images, prompts, and responses).

Design principle. The SAS does not prescribe specific algorithms, models, or data formats at any layer. For example, a deployment using Nav2 at L1, YOLO-based object detection at L2, and Qwen-based VLM reasoning at L3 instantiates the same framework as an alternative deployment using a proprietary planner, semantic segmentation, and a different VLM. The framework’s value lies in defining layer responsibilities and interfaces, enabling systematic comparison of implementations and principled component replacement.

The framework therefore acts not only as a conceptual taxonomy, but also as an integration template: each layer exposes a limited set of inputs and outputs to adjacent layers, allowing perception, reasoning, navigation, and memory modules to evolve independently while preserving system-level behavior.

3.2. Comparison with existing approaches

Table 1 compares the SAS with four categories of existing systems, mapped onto the six-layer structure. The comparison is based on published descriptions and documented capabilities; we note where specific layers are absent or partially addressed.

Commercial AMRs (e.g., OTTO Motors, MiR, KUKA KMR iiwa) provide highly optimized L0–L1 stacks, proprietary perception at L2, and fleet management systems at L5 that track KPIs and coordinate task allocation. They interface with warehouse management systems through industrial protocols such as VDA 5050 at L4. Commercial AMRs provide highly optimized L0–L1 capabilities, proprietary perception pipelines at L2, and fleet-management functions at L5 for task allocation, monitoring, and KPI tracking. They commonly interface with warehouse or production systems through industrial protocols such as VDA 5050 at L4. However, their task execution is typically driven by structured commands, predefined locations, or system-level missions rather than by open-ended semantic interpretation of operator intent. The SAS addresses this gap by introducing L3 as a dedicated layer for resolving natural-language or task-level intent into semantically grounded navigation targets.

Table 1. Comparison of the SAS with existing architectural approaches. A dash (-) indicates that the layer is absent or not addressed in the published system description. “Partial” indicates that some functionality exists but does not cover the full layer capability as defined by the SAS.

Layer	Commercial AMRs	Research VLN	LLM-ROS frameworks	SAS (this work)
L5	Fleet analytics, KPIs	-	Generic memory plugins	M1–M5 with scope taxonomy
L4	VDA 5050, REST APIs	Natural language	Generic tool-calling	NL + TUI + extensible
L3	-	LLM/VLM (full pipeline)	LLM with tool-use	Hybrid L3a/L3b
L2	Proprietary, optimized	Idealized or simulated	Standard ROS topics	ASF + YOLO [prior work]
L1	Optimized, proprietary	Simulated Nav2 or custom	Standard Nav2	Nav2 + Route Server
L0	Commercial platforms	Simulated (AI2-THOR, Habitat)	Any ROS 2 robot	Custom Xplorer (validated)
Cross-robot memory	Fleet-level coordination	Not demonstrated	Not demonstrated	Demonstrated (this work)
Physical validation	Production environments	Predominantly simulation	Lab demonstrations	Multi-robot, multi-session

Research Vision-Language Navigation (VLN) such as those surveyed by Alqobali et al. [4] and exemplified by landmark-based navigation, annotated semantic-map representations, and multi-robot semantic coordination approaches [9,15,23], focus on sophisticated semantic reasoning for navigation. These systems have substantially advanced the ability of agents to interpret language, landmarks, and visual context. However, their evaluation is often benchmark-driven, simulation-based, or focused primarily on the reasoning component rather than on the full physical ROS 2/Nav2 integration chain. The SAS requires L3 reasoning to operate on top of validated L1–L2 navigation and perception infrastructure, under real sensing noise, localization uncertainty, and physical robot constraints.

LLM-ROS frameworks such as ROSClaw [26], ROSA [28], and ROS-LLM [27] provide model-agnostic executive layers for controlling ROS 2 robots through natural-language or tool-calling interfaces. ROSClaw in particular formalizes the executive contract $C = \langle A, O, V, L \rangle$, which we adopt in our L3 implementation. These frameworks are intentionally general-purpose: they expose robot capabilities, observations, validation mechanisms, and logging structures to language-model agents. The SAS is complementary and domain-specific: it specializes this executive-contract perspective for semantic navigation by combining graph-based target resolution, deterministic-before-VLM reasoning, Nav2 Route Server execution, and memory-driven cross-robot transfer. Recent RCIM work has also begun to formulate MLLM-enabled robotic collaboration through perception–decision–execution coordination mechanisms [34]. The SAS follows a related layered philosophy, but specializes it for semantic indoor mobile-robot navigation: perception produces graph-grounded semantic context, reasoning resolves operator intent to a navigation target, and execution is delegated to Nav2 while memory enables cross-session and cross-robot reuse.

3.3. Validated instance

Table 2 maps the SAS layers to our concrete implementation, validated across Sprints 1–7.3 of development on the SAIM Xplorer platform. Layers L0–L2 were validated in our prior work [7] on three physical robots (Xplorer-A, B, and C) across 115 navigation legs with a 97% overall success rate. The present paper focuses on L3 (Section 4) and L5 (Section 5), which constitute the new contributions (Table 2).

Physical deployment architecture. The validated SAS instance is distributed across the mobile robot platforms and a shared operator workstation. Layers L0–L2 run onboard each robot on Raspberry Pi 5 hardware, using CPU-only execution for navigation, perception, and semantic map updates. Layers L3 and L5 run on the operator workstation equipped with an NVIDIA RTX 5050 GPU, where VLM inference, semantic reasoning orchestration, memory extraction, and audit-log analysis are performed. Communication between workstation and robot uses two channels: HTTP requests to the robot context bridge, implemented with FastAPI on port 8080 and serving pose, camera, graph, and semantic-object data; and ROS 2 DDS action calls, including ComputeRoute, FollowPath, NavigateToPose, and Spin, to the robot’s Nav2 stack. This separation reflects the current compute reality: VLM inference in L3b requires GPU acceleration that is not available on the Raspberry Pi 5 platforms, whereas L0–L2 are designed for onboard edge execution. If GPU-capable edge compute such as NVIDIA Jetson Orin is installed onboard, L3 can migrate to the robot without changing the SAS layer interfaces; only the deployment topology changes.

Table 2. SAS instance on the SAIM Xplorer platform. L0–L2 are summarized from prior validated work [7]; L3–L5 are the contributions of this paper.

Layer	Framework capability	Implementation	Validation
L5	Operational intelligence + semantic memory	OpenClaw agent (minimax-m2.7) + M1–M5 JSONL memory with compiled digest (≤ 500 tokens) + VLM logging + audit log analysis. Multi-robot support via <i>platform_id</i> tagging and per-robot M4 generation.	Section 5; Sprint 7.1–7.3
L4	Mission interface	<i>/vlm_instruction</i> ROS 2 topic (std_msgs/String) + OpenClaw terminal user interface	Sprint 3+
L3	Semantic reasoning	Hybrid L3a (7-step deterministic, < 0.1 ms) / L3b (Qwen 3.5:4b via Ollama, 2–9 s) + visual confirmation with 4 signature-type branches + executive contract (A, O, V, L)	Section 4; Sprint 5–7.3
L2	Perception	Monocular C920 + 2D LiDAR STL-19P, YOLO26n (C++ NCNN, 5.5 Hz), Angular Sector Fusion at 3 Hz (< 1 ms/detection), 18 static POIs with attributes in GeoJSON	[7]
L1	Navigation execution	Nav2 Jazzy 1.3.10: AMCL + EKF at 20 Hz, Route Server (24 nodes, 60 edges), MPPI controller, penalty-weighted replanning ($\delta > 1.0$, 10 s interval)	[7]
L0	Mobile platform	Xplorer-B (single RPi5 16 GB) and Xplorer-C (dual RPi5), 4WD differential drive, RoboClaw/ESP32 motor control	[7] for L0–L2; this work for Xplorer-B/C deployment.

4. Hybrid Semantic Reasoning

This section describes Layer 3 (Semantic Reasoning) of the SAS framework, which bridges the gap between human intent expressed in natural language and executable navigation targets in the robot’s graph-based environment representation. The proposed architecture follows a dual-process structure: a fast deterministic path (L3a) resolves unambiguous or previously learned instructions without invoking a language model, while a deliberative path (L3b) activates a Vision-Language Model only when the deterministic cascade cannot produce a confident and unambiguous target.

4.1. Design rationale

The decision to layer deterministic resolution before VLM reasoning was motivated by three empirical observations from preliminary development:

First, the majority of navigation instructions in indoor environments are semantically unambiguous. An instruction such as “go to lab CB204” maps directly to a named node in the navigation graph; invoking a VLM for such an instruction introduces several seconds of avoidable inference latency and temporarily occupies the shared GPU resource. Across the controlled experimental scenarios reported in Section 7, deterministic resolution (L3a) handled 72 of 82 scenario-level decisions (88%) without any VLM involvement.

Second, VLM inference on consumer hardware is inherently variable. Measured inference times on a laptop GPU (NVIDIA RTX 5050) with Qwen 3.5:4b ranged from 2.6 to 8.9 seconds per decision (Section 7.1), with variability attributable to prompt length, context complexity, and GPU scheduling. This latency is acceptable for occasional deliberative decisions but incompatible with continuous operation where multiple instructions may arrive in rapid succession.

Third, the deterministic path does not require a camera image. On the L3a path, no image is fetched from the robot - saving the HTTP transfer latency (200–500 ms) and the bandwidth cost of a 640×480 JPEG frame. The camera is activated on-demand only when L3b is invoked, reducing both network load and power consumption on the edge platform.

4.2. L3a: seven-step deterministic resolver

The deterministic resolver receives a natural language instruction and attempts to match it against the navigation graph structure, static POI attributes, and operator preference patterns. It operates as a cascade of seven steps, evaluated in a fixed order (Fig. 2). The first step that produces an unambiguous match returns immediately; if no step matches, the instruction escalates to L3b.

Linguistic normalization. Before matching, the instruction is normalized: converted to lowercase, underscores replaced with spaces, diacritical marks stripped (supporting Romanian input), and whitespace collapsed. This normalization is applied identically to both the instruction and the graph/POI labels, ensuring consistent matching regardless of input formatting. The cascade contains seven ordered tests, numbered Step 0 to Step 6 because the memory-based preference match is evaluated before the original deterministic graph- and attribute-matching steps.

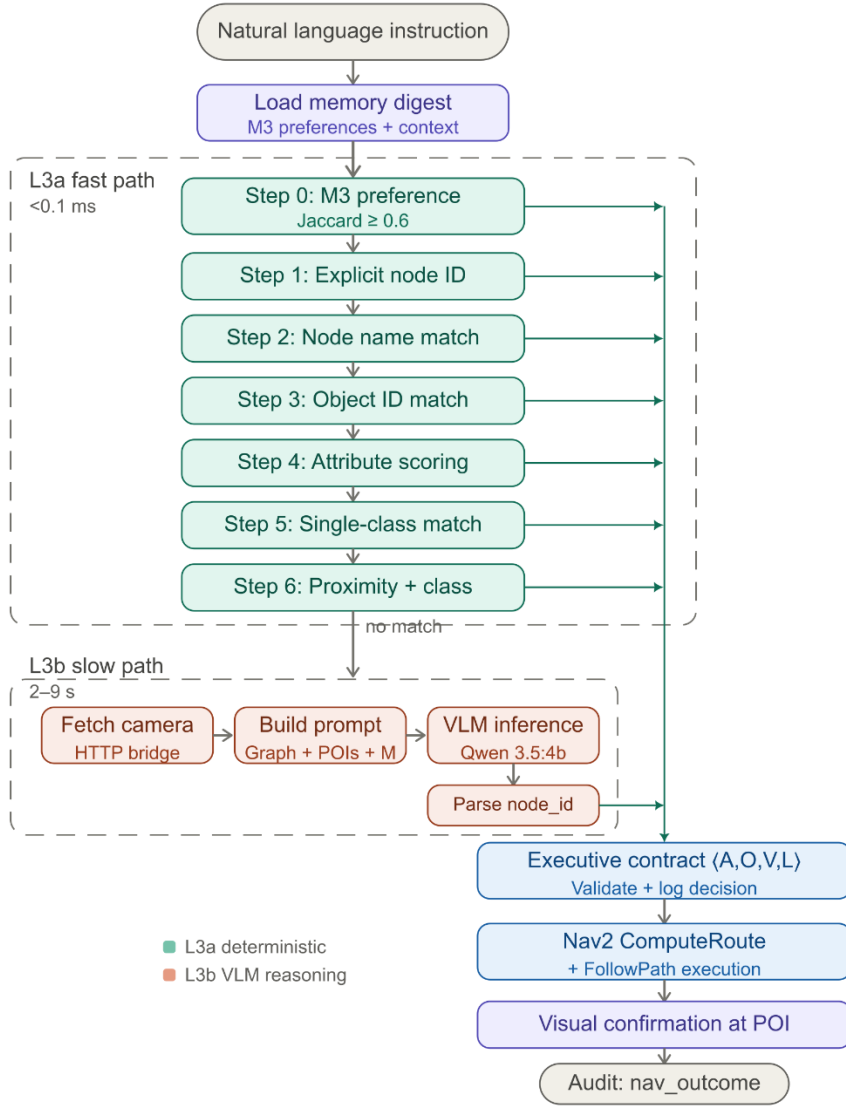


Fig. 2. Hybrid reasoning flowchart. Natural language instructions are first processed by the L3a seven-step deterministic cascade; the first matching step returns immediately. If no step produces an unambiguous resolution, the instruction escalates to L3b, which acquires a camera image, constructs a structured prompt, and invokes the VLM. Both paths converge at the executive contract $\{A, O, V, L\}$, which validates the decision and dispatches Nav2 actions with post-arrival visual confirmation.

Algorithm 1 presents the complete resolver cascade. Each step is described in detail below.

Algorithm 1: L3a Deterministic Instruction Resolver

Input: I (instruction), G (graph with named nodes),
 S (static POIs with attributes),
 P (robot pose: x, y, yaw),
 D (compiled M3 preferences digest)

Output: $(node_id, method)$ or `ESCALATE_TO_L3B`

Step 0 - Preference match (M3):

```

tokens ← tokenize(normalize(I)) \ STOPWORDS
for each preference  $p$  in  $D$ .promotions:
    for each example  $e$  in  $p$ .examples:
         $j \leftarrow \text{jaccard}(\text{tokens}, \text{tokenize}(e))$ 
        if  $j \geq 0.6$ :
            candidates ← candidates  $\cup \{(j, p.\text{node\_id})\}$ 

```

```

    if single unambiguous match:
        return (node_id, "L3a_m3_preference")
Step 1 - Explicit node ID:
    if regex matches "node <N>" or "nodul <N>" in I:
        if N exists in G:
            return (N, "L3a_deterministic")
Step 2 - Node name match:
    matches ← {n ∈ G | normalize(n.name) ⊂ I or I ⊂ normalize(n.name)}
    if |matches| = 1:
        return (matches[0], "L3a_deterministic")
Step 3 - Object ID match:
    matches ← {o ∈ S | normalize(o.obj_id) ⊂ I}
    if |matches| = 1:
        return (o.nearest_node, "L3a_deterministic")
Step 4 - Attribute scoring:
    for each o ∈ S:
        score(o) ← count of attribute values matching words in I
    if single highest-scoring object:
        return (o.nearest_node, "L3a_deterministic")
Step 5 - Single-class match:
    classes ← {c | c.name appears in I, at least one o ∈ S has class c}
    if |classes| = 1:
        objs ← static objects of that class
        if |objs| = 1:
            return (o.nearest_node, "L3a_deterministic")
Step 6 - Proximity + class:
    if proximity keyword in I ("closest", "nearest", "cel mai apropiat"):
        if |classes| = 1 and |objs| > 1:
            nearest ← argmin_{o ∈ objs} distance(P, o)
            return (nearest.nearest_node, "L3a_deterministic")

return ESCALATE_TO_L3B

```

Step 0 - Preference match (M3). This step, introduced as part of the semantic memory framework (Section 5), enables the resolver to recognize instructions that the system has previously encountered and resolved through VLM reasoning. The instruction is tokenized (with English and Romanian stop words removed), and each token set is compared against the stored preference examples using the Jaccard similarity coefficient. A match is accepted when the similarity exceeds the threshold (default: 0.6) and the match is unambiguous (a single node ID among all matching preferences). When multiple preferences produce tied scores pointing to different nodes, the match is rejected and the cascade continues. The threshold of 0.6 was selected during preliminary development and then kept fixed for the controlled experimental sessions reported in Section 7. It was chosen to accept paraphrased versions of the same instruction while rejecting unrelated instructions.

Step 1 - Explicit node ID. A regular expression searches for patterns indicating a node reference (“node 5”, “nodul 14”) in both English and Romanian. If the extracted integer corresponds to a valid node in the graph, it is returned immediately. This step handles operator-level navigation commands.

Step 2 - Node name match. The normalized instruction is tested for substring containment against each node name in the graph. Names with underscores are also tested with spaces replacing underscores (e.g., “lab_cb204” matches “go to lab cb204”). A match is accepted only when exactly one node name matches; multiple matches indicate ambiguity.

Step 3 - Object ID match. Similar to Step 2 but operates on the *obj_id* field of the static POI set rather than on graph node names. This handles instructions that reference specific objects (e.g., “go to plant_3”).

Step 4 - Attribute scoring. For each static POI, the system counts how many explicit attribute values appear lexically in the instruction after normalization. Attributes considered include type, department, seating, use, and custom fields; however, the step does not perform synonym expansion, embedding-based similarity, or affordance inference. Therefore, an instruction such as “sit and relax” does not automatically match a POI with use = resting spot unless the relevant lexical attribute values are explicitly present in the instruction. Such affordance-level mappings are intentionally delegated to L3b and can later be promoted into M3 preferences.

Step 5 - Single-class match. The instruction is scanned for object class names (e.g., “plant”, “radiator”, “laboratory”). If exactly one class is mentioned and that class has exactly one instance in the static POI set (after filtering out YOLO-detected duplicates), the corresponding node is returned. The v4.8 implementation preferentially uses static (surveyed) objects over YOLO-detected objects of the same class to ensure resolution stability.

Step 6 - Proximity + class. Activated only when a proximity keyword (“closest”, “nearest”, “cel mai apropiat”) is present in the instruction AND a single class matches in Step 5 but has multiple instances. The system computes the Euclidean distance from the robot’s current AMCL pose to each instance and returns the nearest. This step was experimentally validated using the instruction “Take me to the closest plant” which correctly resolved to the nearest of five potted plant POIs on both robots (Section 7.3).

Cascade properties. The resolver is deterministic: given the same graph, POI set, pose, and digest, it always produces the same result. It requires no training data, no GPU, and no network access beyond the AMCL pose subscription. When no step produces an unambiguous match, the cascade returns *ESCALATE TO L3B*, triggering VLM reasoning. This explicit escalation - rather than defaulting to the best partial match - prevents the system from acting on uncertain information.

4.3. L3b: VLM visual-semantic reasoning

L3b is activated only when the deterministic cascade exhausts all seven steps without producing an unambiguous resolution. Upon activation, L3b performs three operations that are absent from the L3a path: (i) a camera image is fetched from the robot via the HTTP bridge, (ii) a structured prompt is constructed incorporating the navigation graph, semantic objects, memory context, and the camera frame, and (iii) the prompt and image are sent to a Vision-Language Model for inference.

VLM configuration. The current implementation uses Qwen 3.5:4b served via Ollama (localhost, port 11434) on the operator workstation (NVIDIA RTX 5050). Model parameters are set to *temperature: 0.3* (low variance to favor consistent spatial reasoning), *num_predict: 500*, and *num_ctx: 16384*. The *think: false* option disables the model’s internal chain-of-thought to reduce latency; when chain-of-thought tokens are present in the output (from model versions that ignore this flag), they are stripped before parsing.

Prompt structure. The VLM prompt (constructed by *build_vlm_prompt()*) is organized into six sections:

1. **Memory prefix** (0–500 tokens): top entities and spatial patterns from the compiled memory digest (Section 5), providing cross-session environmental context.
2. **Current state:** robot pose (x, y, yaw) from AMCL.
3. **Navigation graph:** all node IDs with names and coordinates.
4. **Semantic objects by zone:** objects grouped by their nearest graph node, with attributes, source (static/YOLO), and distance from robot. Static objects are labeled as reliable; YOLO objects include confidence scores.
5. **Designated seating/resting locations** (v4.8): a dedicated block listing objects with *seating: true* or *use: resting spot*, placed before the task section to guide affordance-based reasoning.
6. **Task and rules:** the operator instruction, followed by explicit rules including node selection from the graph, preference for static over dynamic objects, and preference for designated seating locations over YOLO-detected chairs.

The VLM is instructed to respond with a JSON object containing *node_id* (integer) and *reason* (string). The response is parsed with fallback handling: markdown code block wrappers are stripped, and if chain-of-thought tags are present, only the content after *</think>* is parsed.

Model agnosticism. The VLM interaction is encapsulated in a single function (*call_vlm()*) that communicates with Ollama’s chat API. Changing the VLM model requires modifying only the *OLLAMA_MODEL* constant (e.g., from *qwen3.5:4b* to *gemma-4-e4b*). The prompt structure, response parsing, and all downstream logic remain unchanged. This design is validated by the executive contract (Section 4.5), which enforces a fixed interface regardless of the model behind it.

4.4. Visual confirmation post-arrival

After the robot reaches the target node coordinates (as reported by FollowPath action completion), the system performs a visual verification step to confirm that the expected objects are present at the destination. This addresses a failure mode common in coordinate-based navigation: the robot may reach the coordinates but face the wrong direction, or AMCL drift may place it at a slightly different location where the target objects are not visible.

The confirmation method is selected based on the *visual_signature_type* attribute assigned to each static POI in the GeoJSON file (Table 3).

Design insight. The signature type classification is based on what the robot’s camera can see at arrival from its typical approach angle and mounting height (16 cm above ground), not on whether objects physically exist at the

POI. For example, the restroom POIs have pictograms on the wall at 1.5 m height - invisible to the forward-facing camera at close range - so they are classified as *sig_type* = *none* and use pose-based confirmation instead. For VLM-based confirmation (landmark and contextual types), the system fetches a camera image at the arrival position, constructs a confirmation prompt listing the expected objects, and queries the VLM. The response includes *at_target* (boolean), *confidence* (HIGH/MEDIUM/LOW), *scene_description*, *reasoning*, and a structured *identified_objects* array listing what the VLM actually observed with position and salience annotations. This structured output (introduced in Task 7.0 schema) enables post-hoc analysis of confirmation failures.

Table 3. Visual confirmation routing by signature type. The signature type encodes what a forward-facing camera is expected to see when the robot arrives at a POI from its typical approach direction.

Signature type	Method	Camera used	Example POIs	Rationale
<i>landmark</i>	VLM verifies named objects	Yes	radiator_main, plant_5	Distinctive objects visible at approach
<i>contextual</i>	VLM verifies neighborhood cues	Yes	plant_2 (near doorway)	Object itself may not be salient; context identifies location
<i>architectural</i>	VLM verifies spatial layout	Yes	corridor intersection (future)	No specific object; layout structure confirms location
<i>none</i>	AMCL pose + radius check	No	restroom_men, restroom_women	No visual signature from camera height (wall-mounted pictograms, recessed doors)

For pose-based confirmation (*none* type), the system computes the Euclidean distance between the robot’s AMCL pose and the target node coordinates. If the distance is within the configured radius (default: 1.5 m, loaded from *policy.yaml*), the arrival is confirmed. This method still captures a finish image for audit purposes but does not invoke the VLM.

The confirmation result does not affect the navigation outcome reported in the audit log - a mission is recorded as *mission_complete* based on Nav2 FollowPath success regardless of confirmation. The confirmation data is logged as a separate field (*confirmation*) in the audit entry, enabling independent analysis of confirmation accuracy.

4.5. Executive contract {A, O, V, L}

The interaction between the VLM and the robot’s navigation capabilities is formalized through an executive contract, adopted from the ROSClaw framework for agentic robot control [26] and specialized for semantic navigation. The contract defines four components:

[A] Affordance manifest. Constructed at each decision cycle from three sources: the navigation graph nodes (IDs, names, coordinates), the merged semantic objects (static POIs + filtered YOLO detections with attributes), and the allowed actions from the safety policy (ComputeRoute, FollowPath, NavigateToPose, Spin). The manifest defines the admissible action and target space exposed to the reasoning layer; the VLM is not allowed to generate arbitrary ROS commands but must select a target and action from the provided affordances.

[O] Observation normalizer. Consolidates everything the robot “sees” into a consistent snapshot: AMCL pose (x, y, yaw), semantic object inventory (counts of static and dynamic objects, each with nearest node mapping), and optionally a camera image (base64-encoded JPEG, present only on the L3b path). The normalizer decouples the VLM from the specific sensor suite - whether the pose comes from AMCL, visual odometry, or RTK GPS, the contract delivers the same structure.

[V] Action validator. Before any navigation action is executed, the proposed action is checked against the safety policy defined in *policy.yaml*. Three checks are performed in sequence: (1) the action type must be in the allowlist (e.g., ComputeRoute is allowed, arbitrary topic publishing is not); (2) the target node must exist in the current navigation graph; (3) the goal distance must be within the configured maximum (default: 50 m). If any check fails, the action is blocked and the failure is logged with the specific check that triggered the rejection. In the experimental data, no actions were blocked - all proposed nodes were valid - but the validator provides a safety boundary against VLM hallucinations (e.g., a non-existent node ID).

[L] Audit logger. Every decision is recorded as a structured JSONL entry containing: timestamp, instruction, resolution method, target node, validation result, navigation outcome, timing breakdown (resolve, VLM inference, navigation total), confirmation data, images (start and finish), and platform-specific metrics (AMCL poses, odometry distance, YOLO diagnostics, battery state). The logger accepts a *platform_id* parameter, enabling distinct per-robot attribution in multi-robot deployments. Session-level events (startup, shutdown) are logged with digest hash and configuration snapshots for reproducibility.

Contract benefits. The {A, O, V, L} contract provides three guarantees: (1) model agnosticism - swapping the VLM model changes only the inference call, not the contract interface; (2) safety boundaries - the validator prevents acting on invalid VLM outputs; and (3) complete audit trail - every decision is traceable from instruction

to outcome with full context. In the experimental evaluation, the contract logged 82 Phase 2 decisions with zero missing fields and full cross-robot traceability.

4.6. Physical deployment architecture

The L3 layer is deployed on the operator workstation (laptop with NVIDIA RTX 5050, Ubuntu 24.04, ROS 2 Jazzy), physically separate from the robot platforms that run L0–L2. Two communication channels connect the workstation to each robot:

HTTP (robot → workstation). Each robot runs a FastAPI bridge on port 8080 that serves context data (AMCL pose, navigation graph, semantic objects, camera image) as JSON endpoints. The workstation’s navigator fetches this data at the beginning of each decision cycle. The bridge is stateless - it returns current sensor readings with no session tracking - enabling the same navigator instance to switch between robots by changing the `CONTEXT_SERVER_URL` environment variable.

ROS 2 DDS (workstation → robot). The navigator uses Nav2 action clients (ComputeRoute, FollowPath, Spin) to command the robot’s navigation stack. DDS operates on a shared domain ID (default: 67) for sequential operation, or on per-robot domain IDs for concurrent operation (default and 67 in the experiments).

The mission interface (L4) is decoupled from both the robot and the workstation: instructions arrive via a ROS 2 topic that any DDS-connected client can publish to, enabling integration with web dashboards, mobile applications, or industrial fleet managers (VDA 5050) without modifying the reasoning or navigation layers. In the validated instance, the operator interacts through an OpenClaw agent profile that provides six workflows - instruction dispatch, emergency stop, mission history, system health, position query, and live status - communicating exclusively through the `/vlm_instruction` topic and structured audit log queries.

Multi-robot operation is achieved through environment variable configuration:

- **Sequential:** Change `XPLORER_PLATFORM_ID` (which auto-selects the corresponding `CONTEXT_SERVER_URL` from `PLATFORM_PROFILES`) between sessions.
- **Concurrent:** Run two navigator instances in separate terminals, each with a different platform ID and ROS domain ID. Both instances read the same memory digest file (read-only at runtime). The shared Ollama instance serializes L3b requests - if both robots escalate simultaneously, one waits while the other completes inference. In practice, M3 preference resolution (L3a) completes in under 0.1 ms, so simultaneous L3b escalation is rare.

This deployment architecture is transitional: it reflects the current compute reality where consumer GPUs capable of VLM inference are not available on the RPi5 robot platforms. As GPU-capable edge compute (e.g., NVIDIA Jetson Orin) becomes available on-robot, L3 can migrate to the platform with no changes to the layer interfaces - only the deployment topology changes, with the HTTP bridge becoming localhost calls and DDS actions remaining identical.

5. Semantic Memory Framework

This section describes Layer 5 (Operational Intelligence) of the SAS framework - the persistent knowledge layer that enables the system to learn from experience, adapt its reasoning over time, and transfer acquired knowledge across robot platforms. The framework organizes semantic memory into five categories (M1–M5) with an explicit scope taxonomy that determines which knowledge is shared globally and which is attributed per-robot. A compiled digest mechanism distills the accumulated knowledge into a compact representation (≤ 500 tokens) that is injected into the VLM prompt at runtime or matched against at L3a Step 0 (Section 4.2).

5.1. Memory categories and scope taxonomy

The memory framework partitions operational knowledge into five categories, each with a defined scope, update source, and sharing policy (Table 4, Fig. 3). The taxonomy is motivated by a fundamental observation in multi-robot deployments: some knowledge describes the environment (shareable across all robots on the same map), some describes the operator (shareable when the same operator commands different robots), and some describes the robot itself (unique per platform).

Table 4. Memory categories with scope taxonomy. Global categories (M1–M3) are shared across all robots operating on the same map with the same operator. Per-robot categories (M4, M5) are attributed by `platform_id`.

Category	Content	Scope	Sharing policy	Update source	Storage
M1 Environment	Entity visit statistics: visit count, success rate, mean navigation time, confidence, position distribution	Global (per-map)	All robots on same map	Audit log: decision entries with <code>node_id</code> and confirmation	JSONL, one record per POI

M2 Temporal patterns	Recurring visual observations clustered by keyword overlap (Jaccard ≥ 0.5), per node	Global (per-map)	All robots	Audit log: confirmation.scene_context text	JSONL, one record per pattern
M3 Operator preferences	Instruction $\rightarrow$ node_id mappings with frequency, consistency, and method distribution	Global (per-operator)	All robots with same operator	Audit log: instruction + node_id + resolution_method	JSONL, one record per instruction group
M4 Platform capabilities	Hardware configuration, software version, sensor suite, compute architecture	Per-robot	Not shared	navigator_startup events + static GeoJSON	JSONL, one file per platform
M5 Task history	Per-decision summary: instruction, resolution, outcome, timing, anomalies	Per-robot (aggregable)	Attributed by platform_id	All decision events	JSONL, all decisions

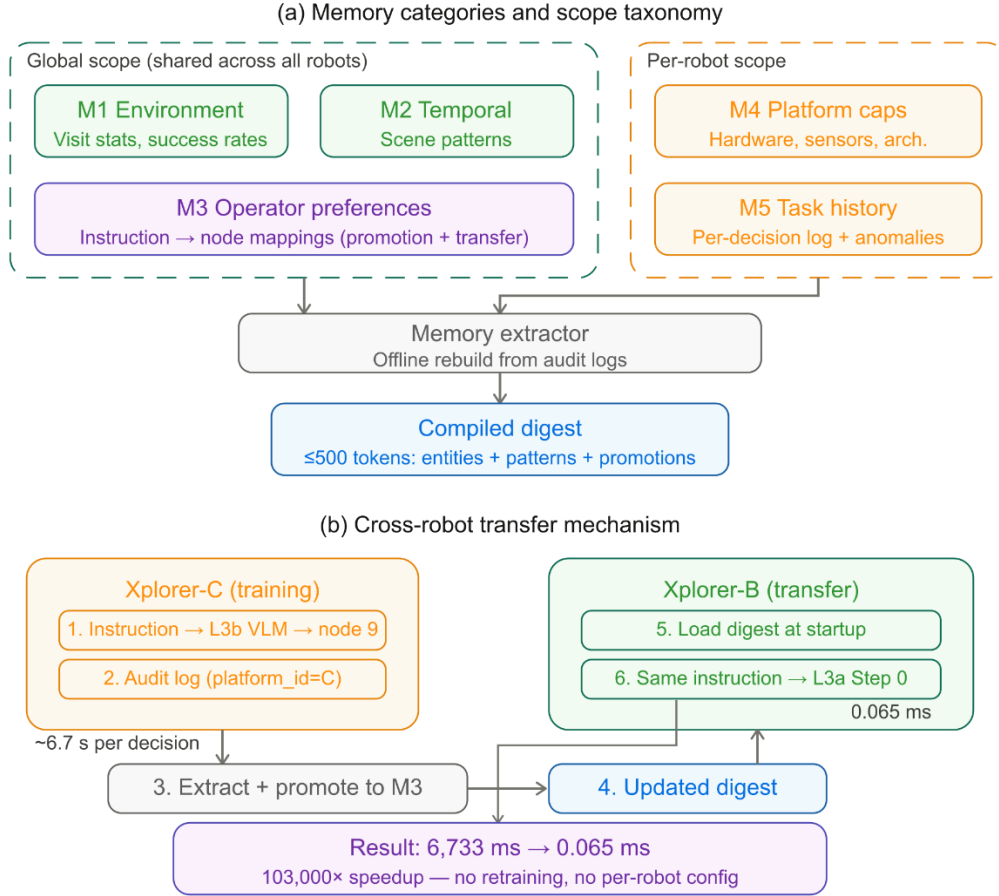


Fig. 3. (a) M1–M5 memory scope taxonomy with global and per-robot partitioning. (b) Cross-robot transfer: VLM-learned preference on Xplorer-C is promoted, compiled into shared digest, and resolved deterministically on Xplorer-B (6,733 ms $\rightarrow$ 0.065 ms).

Design rationale. The five categories were derived from analyzing what knowledge would be useful to share vs. what would be misleading to share across robots:

- M1 and M2 describe the environment. If a corridor section consistently causes navigation failures (M1) or always contains the same objects (M2), this is true regardless of which robot operates there. Sharing these observations across robots can help the system anticipate known problematic locations and support more informed target selection or post-hoc analysis, while still preserving per-robot performance attribution through M4 and M5.

- M3 describes the operator’s intent patterns. When an operator consistently uses “take me somewhere for fresh air” to mean node 14 (window), this preference applies regardless of which robot receives the instruction. M3 is the category that enables the cross-robot transfer demonstrated in this work.
- M4 describes the robot. Xplorer-B has a single RPi5, Xplorer-C has dual RPi5 with inference offloading - these capabilities differ and should not be merged. M4 enables the system to record platform-specific constraints that may affect future task allocation.
- M5 records operational history. Each decision is tagged with *platform_id*, enabling per-robot performance analysis while remaining aggregable for global statistics.

5.2. Memory extractor

The memory extractor (*memory_extractor.py*, 716 lines) is an offline post-processing tool that reads all audit JSONL files from the logs directory and produces the five memory files plus a compiled digest. It operates in snapshot rebuild mode: each execution regenerates all outputs from scratch, ensuring idempotency and simplifying debugging.

Input. Audit JSONL files produced by the navigator (v4.7.2+), each containing session_start, navigator_startup, and decision entries. The extractor identifies platform attribution from navigator_startup events and groups sessions accordingly.

M1 extraction. For each static POI in the GeoJSON, the extractor counts how many decision entries targeted that node, computes the success rate (proportion with *confirmation.confirmed = true*), and calculates mean navigation time and confidence. A per-platform breakdown is included when the *--platforms* flag is specified. POIs with fewer than 2 visits are marked as *unreliable*.

M2 extraction. The scene_context text from visual confirmation results is collected per target node. Within each node, observations are clustered by keyword overlap using Jaccard similarity (threshold ≥ 0.5). Clusters with at least 3 observations and confidence ≥ 0.6 are emitted as temporal patterns. Each pattern contains a keyword-based description (e.g., “floor near plant tiled window”), enabling the VLM to recognize familiar scenes.

M3 extraction and promotion. This is the core mechanism that enables the learning cycle demonstrated in this work. For each unique instruction (after normalization: lowercase, stop words removed, tokens sorted), the extractor tracks: - **Frequency:** how many times the instruction was given (across all sessions and platforms) - **Dominant node:** the most frequently resolved target node - **Consistency:** proportion of resolutions to the dominant node (e.g., $6/7 = 0.86$) - **Method distribution:** counts of L3a_deterministic, L3a_m3_preference, L3b_vlm

A preference is **promoted to L3a** (marked *ready_for_l3a_promotion = true*) when all three conditions are met:

1. *frequency* ≥ 3 (the instruction has been given at least 3 times)
2. *consistency* ≥ 0.80 (at least 80% of resolutions agree on the same node)
3. *method_counts['L3b_vlm']* ≥ 1 (the instruction was resolved by the VLM at least once)

Condition 3 ensures that only instructions that genuinely required semantic reasoning are promoted. An instruction like “go to node 5” resolves deterministically and should not create an M3 preference - it would be redundant with L3a Step 1. Only instructions that the VLM had to reason about (mapping affordances, context, or intent to a specific node) become preferences.

M4 extraction. For each platform ID specified in *--platforms*, the extractor produces a capability record from the latest navigator_startup event: navigator version, VLM model, policy path, POI signature counts, and architecture description.

M5 extraction. Each decision is distilled into a compact summary: instruction, resolution method, target node, outcome, timing, and anomaly flags (VLM parse errors, reroutes, failed confirmations). The *platform_id* is preserved for per-robot attribution.

5.3. Compiled digest

The digest is the runtime artifact that carries cross-session knowledge into the navigator. It is a single JSON file compiled from M1–M5, constrained to approximately 500 tokens (≤ 2000 characters) to fit within the VLM context window alongside the prompt.

The digest contains four sections:

1. **Top entities** (from M1): the 5 most-visited POIs with visit counts, success rates, and signature types. Injected into the L3b prompt as “OBSERVED ENVIRONMENT” prefix, giving the VLM awareness of which locations the robot has visited frequently and how reliably.
2. **Top patterns** (from M2): the 3 highest-confidence spatial patterns. Provide scene recognition context (e.g., “near node 20, the camera typically sees floor, plant, tiled window”).
3. **L3a promotions** (from M3): the promoted instruction→node mappings. These are consumed by L3a Step 0 (Section 4.2) via Jaccard matching - the critical mechanism that converts learned VLM knowledge into deterministic, sub-millisecond resolution.
4. **Global statistics** (from M5): total task count and overall success rate. Provides the VLM with a sense of system reliability.

Size management. If the serialized digest exceeds the 2000-character limit, entities are trimmed to 3 and patterns to 2. Promotions are never trimmed - they are the highest-priority content.

5.4. Runtime integration

The digest is loaded once at navigator startup and integrated into the reasoning pipeline through two channels:

Channel 1: L3a Step 0 (M3 preferences). The promoted preferences from the digest are compiled into an in-memory index (*m3_index*). Each preference's instruction examples are tokenized (with stop words removed). When a new instruction arrives, it is tokenized and compared against all preference examples using Jaccard similarity. A match above the threshold (0.6) that uniquely identifies a single node triggers immediate L3a resolution - bypassing Steps 1–6, the VLM, and the camera entirely.

This is where the learning cycle completes: an instruction that initially required 6.7 seconds of VLM reasoning on one robot is resolved in 0.065 milliseconds on any robot that shares the digest. The speedup of approximately 103,000× measured in the experiments (Section 7.2) is a direct consequence of this mechanism.

Channel 2: L3b prompt prefix (M1 + M2). When L3a cannot resolve and escalates to L3b, the top entities and patterns from the digest are injected as a prefix block in the VLM prompt (function *build_memory_prefix()*). This provides the VLM with cross-session environmental awareness: which locations are frequently visited, which have high or low success rates, and what visual patterns are typical at each node.

Fault tolerance. If the digest file is missing, corrupted, or empty, the navigator logs a warning and continues without memory. L3a Step 0 is skipped (no M3 index), and the L3b prompt has no memory prefix. All other functionality - Steps 1–6, VLM reasoning, visual confirmation, executive contract - operates unchanged. This design ensures that memory is an enhancement, not a dependency.

5.5. Cross-robot memory architecture

The memory framework is designed to support multi-robot deployments where multiple robots share an environment and an operator. The cross-robot architecture has three properties:

Single source of truth. All memory files reside on the operator workstation (*memory/*). There is no per-robot memory storage. The digest file is read by every navigator instance at startup (read-only at runtime). This eliminates synchronization conflicts - there is exactly one version of each memory file at any time.

Platform-aware extraction. When the extractor runs with *--platforms xplorer-b,xplorer-c*, it demultiplexes audit entries by *platform_id* and produces per-robot M4 files (*M4_xplorer-b.jsonl*, *M4_xplorer-c.jsonl*) and per-robot breakdowns in M1 (*visit_count_by_platform*, *success_rate_by_platform*). The global categories (M1, M2, M3) aggregate data from all platforms - a preference learned on Xplorer-C through VLM interactions appears in the shared digest and becomes available to Xplorer-B on next startup.

Transfer mechanism. Cross-robot transfer is achieved through the shared digest: (1) operator gives an instruction on Robot C → (2) L3b VLM resolves it → (3) decision logged in audit with *platform_id=xplorer-c* → (4) memory extractor runs → (5) M3 promotion criteria met → (6) preference appears in *l3a_promotions_ready* in digest → (7) Robot B loads the same digest at startup → (8) same instruction on B matches via Jaccard at L3a Step 0 → (9) deterministic resolution in <0.1 ms, no VLM, no camera. No code changes, no per-robot configuration, no retraining. Between logging the original VLM-mediated decision and reusing the preference on another robot, the only offline processing step is the memory-refresh procedure, which validates the logs, rebuilds the memory files, and regenerates the compiled digest.

5.6. Update strategy and reproducibility

Current implementation (Sprint 7.3). Memory updates are performed manually between experimental sessions by running *refresh_memory.sh*, which executes three steps: (1) validate audit logs against schema v2.0, (2) backup current digest and run the extractor with platform demultiplexing, (3) display the diff between old and new digest and record the MD5 hash. This manual approach was chosen deliberately for experimental reproducibility: each experimental session operates under a known, documented digest state, and the experimenter controls exactly when knowledge is incorporated.

Reproducibility guarantees. The navigator logs the digest MD5 hash in every startup event. The extractor is idempotent (same inputs → same outputs). The *refresh_memory.sh* script records the hash after each extraction. Combined, these mechanisms allow any experimental session to be traced to its exact memory state.

Future evolution (planned). The present paper validates the offline digest-based memory architecture. For larger fleet deployments, the same M1–M5 taxonomy could be implemented through a shared memory server: robots push audit logs asynchronously via REST API, the server runs the extractor periodically, and robots pull updated digests at startup with a local cache fallback. The M1–M5 scope taxonomy was designed with this transition in mind - global categories (M1–M3) are served to all robots, while per-robot categories (M4, M5) are served only to the requesting platform. The digest compilation and promotion logic remain unchanged.

6. Experimental Methodology

This section describes the controlled physical-robot experimental protocol used to validate the SAS framework contributions. The evaluation is structured as a three-session experiment on two physical robots, designed to demonstrate: (i) the learning cycle from VLM reasoning to deterministic resolution (Session A), (ii) cross-robot memory transfer through a shared compiled digest (Session B), and (iii) concurrent multi-robot operation feasibility without direct robot-to-robot communication (Session C).

6.1. Robotic platforms

Two custom-built differential-drive robots from the SAIM Xplorer platform family were used (Fig. 4), both running the full SAS stack described in Sections 3–5.



Fig. 4. *Xplorer-B (left) and Xplorer-C (right) robotic platforms.*

Both platforms run the same high-level ROS 2 Jazzy/Nav2 semantic-navigation stack, including the same navigation graph, static POI annotations, route-server configuration, and L3/L5 reasoning interface. Their low-level hardware differs in compute distribution, drivetrain electronics, encoder type, and IMU availability, as summarized in Table 5. These differences are intentionally retained because the experiment evaluates whether semantic reasoning and memory transfer remain consistent across non-identical physical platforms sharing the same map and semantic annotations.

Table 5. *Platform specifications. Both robots share the same sensor suite and software stack above L0. The key architectural difference is the compute distribution for YOLO inference.*

Component	Xplorer-B	Xplorer-C
Compute (navigation)	Raspberry Pi 5, 16 GB, ARM Cortex-A76, Ubuntu 24.04	Same
Compute (inference)	Same RPi5 (single-board)	2nd RPi5 via Ethernet (dual-board)
Camera	Logitech C920 HD, configured at 640×480 , USB	Same
2D LiDAR	STL-19P (LD19), 360° , ~ 503 rays, 10 Hz	Same
IMU	BNO055, 9-DoF, I2C	-
Drivetrain	4WD differential, goBILDA, quadrature encoders	4WD differential, hall encoders
Motor controllers	2× RoboClaw 2 × 15A, UART	2× L298N + ESP32
Architecture label	<i>single-rpi5</i>	<i>dual-rpi5</i>
VLM workstation	Laptop: Intel Core Ultra 7 255H, NVIDIA RTX 5050, 32 GB RAM, Ubuntu 24.04, ROS 2 Jazzy	Same (shared)

6.2. Environment and semantic annotations

Experiments were conducted in the second-floor corridor of the Faculty of Industrial Engineering and Robotics (FIIR), National University of Science and Technology POLITEHNICA Bucharest - the same environment used in the prior Sensors study [7].

The navigation graph comprises 24 nodes and 60 directed edges (30 bidirectional pairs), encoded as a GeoJSON file (Fig. 5 and Table 6).

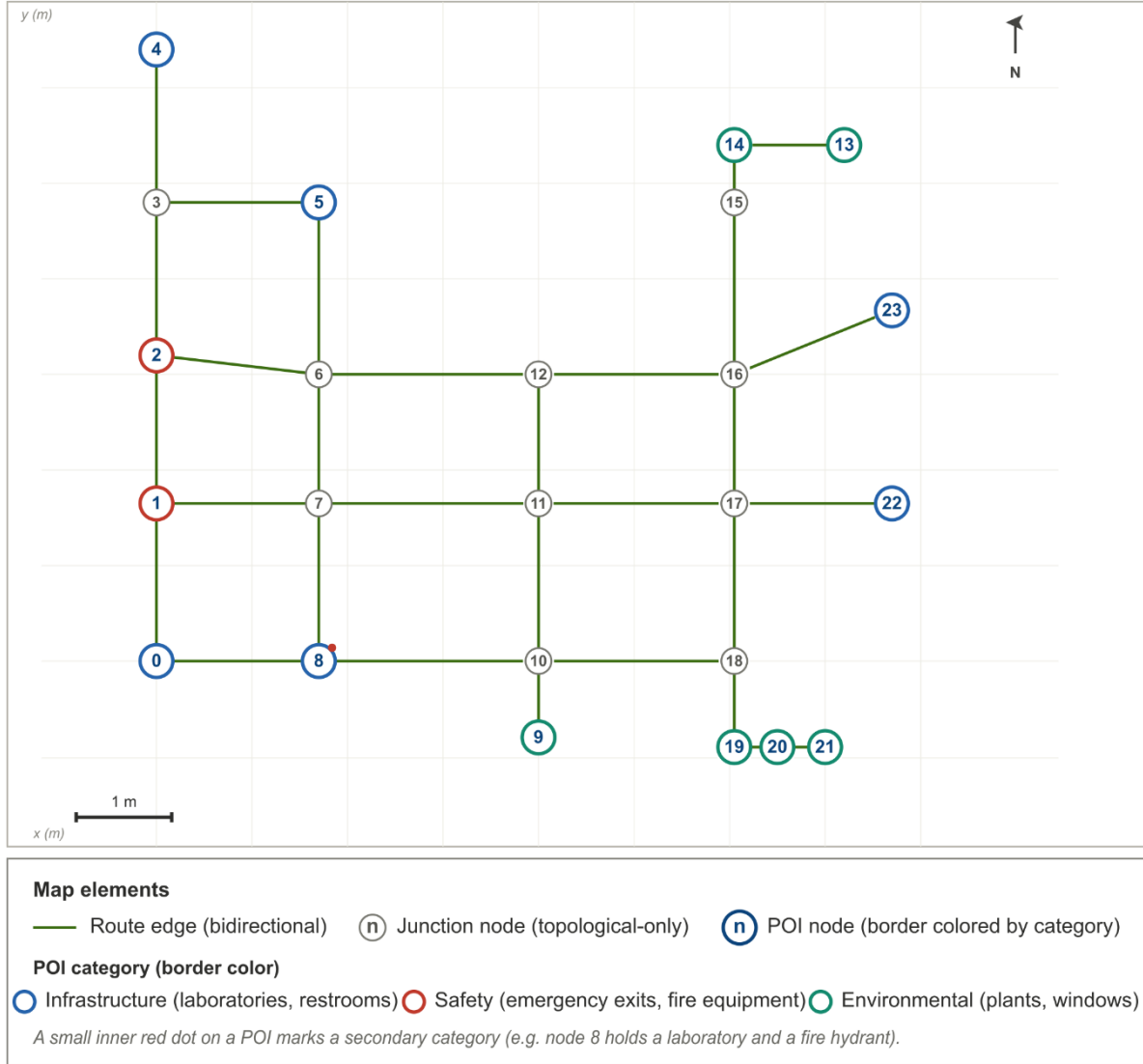


Fig. 5. Navigation graph topology for the FIIR corridor environment. The graph comprises 24 nodes and 60 directed edges (30 bidirectional pairs). POI nodes (bold border) are color-coded by semantic category: blue - infrastructure, red - safety, green - environmental. Junction-only nodes are shown in grey.

Table 6. POI-bearing nodes in the FIIR corridor navigation graph. Each of the 14 nodes hosts one or more of the 18 static semantic objects summarized in Table 7. The Sig. Type column indicates the post-arrival confirmation method (Section 4.4).

ID	Node Name	Adjacent Object(s)	Class	Sig. Type	Category
0	toilet_m	restroom_men	restroom	none	infrastructure
1	hall_entry	hall_entrance	emergency_exit	landmark	safety
2	hall_exit	hall_exit	emergency_exit	landmark	safety
4	toilet_f	restroom_women	restroom	none	infrastructure
5	cb204	lab_cb204	laboratory	landmark	infrastructure
8	cb202	lab_cb202, fire_hydrant_cb202	laboratory + hydrant	landmark	infra. + safety
9	radiator	radiator_main	radiator	landmark	environmental
13	plant_4	plant_4	potted plant	landmark	environmental
14	plant_5	plant_5, window_north	potted plant + window	landmark	environmental
19	plant_3	plant_3, window_south	potted plant + window	landmark	environmental
20	plant_2	plant_2	potted plant	landmark	environmental

21	plant_1	plant_1	potted plant	landmark	environmental
22	cb203_entrance	lab_cb203_entrance	laboratory	landmark	infrastructure
23	cb203_exit	lab_cb203_exit	laboratory	landmark	infrastructure

The environment is annotated with 18 static Points of Interest (POIs) across 8 object classes (Table 7). Each POI has a *visual_signature_type* attribute that determines the post-arrival confirmation method (Section 4.4) and a set of semantic attributes used by the L3a attribute matching step (Section 4.2, Step 4).

Table 7. Static POI summary. The 18 POIs are manually surveyed and encoded in `semantic_objects_static.geojson`. Attributes support L3a matching (Step 4) and L3b prompt construction.

Class	Count	Signature types	Key attributes
potted plant	5	4 landmark, 1 contextual	near_window
laboratory	4	4 landmark	room_number, department, access, entrance
restroom	2	2 none	type (men/women)
emergency_exit	2	2 landmark	type (entrance/exit)
window	2	2 landmark	opens, side (north/south)
fire hydrant	1	1 landmark	type (wall_mounted)
radiator	1	1 landmark	seating=true, use=resting spot
chair	1	1 landmark	seating=true, use=resting spot
Total	18	15 landmark, 2 none, 1 contextual	

6.3. Pre-experiment preparation

Phase 0 - Archive. All development-phase audit logs (Sprints 5–7.2) were archived to a separate directory. This established a clean separation between exploratory development data and controlled experimental data.

Phase 1 - Seed. Three structured seed sessions were conducted on Xplorer-C prior to the experiment, generating 15 L3b VLM decisions across three semantic instructions: S1 (restroom → node 0), S2 (fire safety → node 8), and S3old (fresh air → node 14). All three instructions met the M3 promotion criteria (frequency ≥ 3 , consistency ≥ 0.80 , L3b count ≥ 1) and were promoted to the initial memory digest.

Digest baseline. Digest states. Two digest states were used during the controlled experiment. The initial digest used at the start of Session A contained five pre-existing M3 preferences: the three seed-promoted preferences (S1, S2, S3old) and two additional preferences promoted from earlier development sessions (“go to the female ward” → node 4, “go to the male ward” → node 0). During Session A, the S3new instruction (“Take me somewhere I can sit and relax” → node 9) was repeatedly resolved through L3b and then promoted by the memory extractor. The resulting six-preference digest was frozen after Session A extraction (MD5: 97241265) and used without modification for Sessions B and C.

6.4. Experimental design

The experiment follows a three-session structure where each session tests a distinct aspect of the framework (Table 8). The sessions were executed in a fixed order because the digest state evolves between sessions: Session A generates the S3new learning data, the memory extractor promotes it, and Sessions B and C use the resulting digest.

Table 8. Experimental sessions and scenarios. *N* indicates the number of scenario-level decisions collected (excluding return navigation commands).

Session	Robot	Purpose	Dates	Navigator	Scenarios
A	Xplorer-C	M3 confirmation + S3new learning cycle	Apr 22–23, 2026	v4.7.4 → v4.8	S1(6), S2(5), S3old(5), S3new(7), S4(3), S5(11)
B	Xplorer-B	Cross-robot transfer validation	Apr 23, 2026	v4.8	S1(14), S2(10), S3old(6), S3new(3), S4(5), S5(3)
C	Both	Concurrent operation feasibility	Apr 24, 2026	v4.8	2 pairs $\times$ 2 robots = 4
Total					82 scenario decisions

Scenario descriptions:

- **S1 (restroom):** “go to a place to take a short break for personal needs” → expected L3a_m3_preference → node 0. Tests transfer of seed-promoted preference.
- **S2 (fire safety):** “the washroom got on fire, help me find something to stop the fire” → expected L3a_m3_preference → node 8. Tests transfer of seed-promoted preference.
- **S3old (fresh air):** “It is too hot in here, take me somewhere I can get some fresh air” → expected L3a_m3_preference → node 14. Seed-promoted preference, tested alongside S1 and S2 for transfer confirmation.
- **S3new (learning cycle):** “Take me somewhere I can sit and relax” → expected L3b_vlm on C (learning), L3a_m3_preference on B (after promotion). Demonstrates the complete L3b → promotion

→ L3a → transfer pipeline. This instruction was selected because L3a cannot resolve it - it requires affordance reasoning about seating attributes that no deterministic step matches.

- **S4 (deterministic control):** “go to lab_cb204” → expected L3a_deterministic (Step 2, node name match) → node 5. Cross-robot consistency control.
- **S5 (proximity):** “Take me to the closest plant” → expected L3a_deterministic (Step 6, proximity + class) → varies by pose. Cross-robot consistency control.

Session C concurrent protocol. Both robots were started on separate ROS 2 DDS domain IDs (Xplorer-B on the default domain, Xplorer-C on domain 67) to isolate their Nav2 stacks. Two navigator instances on the workstation - one per robot - shared the same memory digest file (read-only). Robots were placed at opposite ends of the corridor and given different instructions navigating to different destinations, avoiding physical collision. Pair 1 tested M3 preference resolution (B: S2 → node 8, C: S1 → node 0); Pair 2 tested deterministic resolution (B: cb203 entrance → node 22, C: lab_cb204 → node 5).

Session C was designed to test concurrent execution of independently resolved missions under shared-memory conditions. It did not test simultaneous L3b escalation or concurrent VLM inference, because all four concurrent decisions were resolved through L3a. The implications of serialized VLM access in the current workstation-based deployment are discussed as a limitation in Section 8.

Navigator versioning. The reported quantitative results use the corrected navigator v4.8 implementation. An earlier version (v4.7.4) was used during initial Session A development runs; these pre-correction entries are retained in the public dataset for transparency but excluded from the main quantitative analysis. The correction and its rationale are described in Section 7.1.

6.5. Metrics

Two categories of metrics were collected, all logged centrally in structured JSONL audit files on the workstation (Table 9). No separate data collection process was required on the robots - the navigator subscribes to robot topics (AMCL pose, odometry, YOLO diagnostics, battery state) via ROS 2 DDS and logs all data in each decision entry.

Table 9. Metrics collected per decision.

Category	Metric	Source	Unit
Semantic reasoning	Resolution method	Audit: resolution_method	L3a_m3 / L3a_det / L3b_vlm
	Resolve time	Audit: timing.resolve_ms	ms
	VLM inference time	Audit: timing.vlm_ms	ms (0 if L3a)
	M3 match info	Audit: extra.m3_match_info	Jaccard score, frequency
Navigation	Outcome	Audit: nav_outcome	mission_complete / missed
	Duration	Audit: timing.nav_total_s	s
	XY error at goal	Audit: extra.xy_error_m	m (AMCL vs target)
	Distance traveled	Audit: extra.distance_traveled_m	m (odometry)
	Start/end pose	Audit: extra.start_pose, end_pose	[x, y, θ]
Confirmation	Method	Audit: confirmation.confirmation_method	vlm_landmark / pose_based
	Result	Audit: confirmation.confirmed	boolean
	Identified objects	Audit: confirmation.payload	structured array
Perception	YOLO FPS	Audit: extra.yolo.fps	Hz
	YOLO inference	Audit: extra.yolo.inference_ms	ms
System	Battery state	Audit: extra.battery	V, A, % (Xplorer-B)
Reproducibility	Digest hash	Startup: memory_digest_hash	MD5
	Navigator version	Startup: version	string
	Platform ID	Every entry: platform_id	xplorer-b / xplorer-c

The separation between resolution metrics, navigation-completion metrics, and semantic-confirmation metrics allows the analysis to distinguish between three different failure modes: incorrect target selection, failure to physically reach the selected target, and failure to visually or pose-wise confirm arrival.

6.6. Statistical methods

The following statistical tests were pre-specified before data collection:

- **Shapiro–Wilk test** for normality assessment on all timing distributions ($\alpha = 0.05$).
- **Mann–Whitney U test** for comparing L3b VLM inference times (Xplorer-C) against L3a M3 preference resolve times (Xplorer-B), with the one-sided alternative hypothesis that L3b times are greater.
- **Cohen’s d** for effect size quantification of the timing difference.
- **Exact binomial test (Clopper–Pearson)** for the 95% confidence interval on the M3 transfer success rate.
- **Fisher’s exact test** for comparing navigation success rates between M3-resolved and deterministic-resolved scenarios.

All tests use $\alpha = 0.05$. Non-parametric tests (Mann–Whitney, Fisher’s exact) were selected as the primary comparison methods because the Shapiro–Wilk test was expected to reject normality for at least some timing distributions, consistent with the findings of the prior Sensors study on the same platforms.

6.7. Experimental procedure

Each mission followed a standardized procedure: (1) verify navigator readiness, (2) publish the instruction via ROS 2 topic, (3) observe robot navigation without intervention, (4) wait for outcome, (5) execute a return navigation command to reposition the robot at sufficient distance from the target (>2 m) for the next run. Return commands (“go to cb204”, “go to toilet m”) generate audit entries that are excluded from the scenario analysis by instruction text filtering.

A live monitoring tool (*session_monitor.py*) running on a separate terminal tracked each decision in real time, classifying instructions by scenario and generating per-session CSV summaries. This eliminated manual note-taking and ensured no decisions were missed or misclassified.

Sessions could be interrupted and resumed without data loss: the navigator writes each decision entry immediately (JSONL append with flush), and the monitor auto-detects new audit files on restart. Multiple audit files per session were permitted and documented in the experiment manifest.

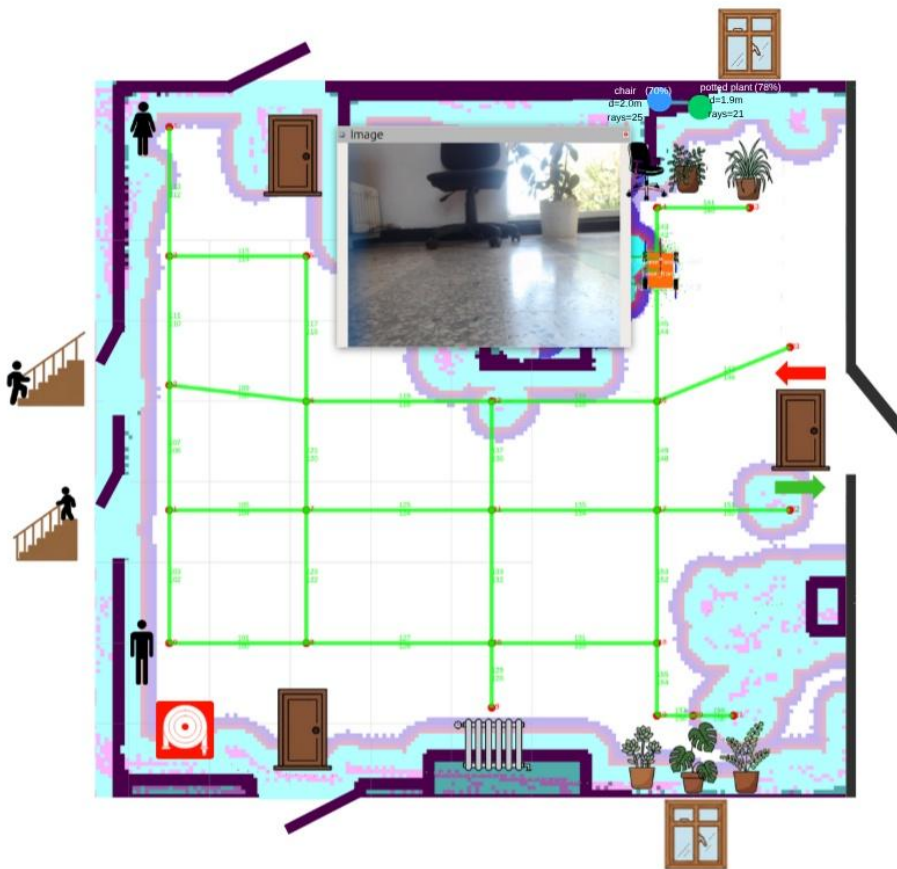


Fig. 6. Live operational view used during the experimental sessions, showing the Nav2 costmap, route graph overlay, 2D LiDAR scan, and real-time YOLO26n detections fused into the semantic-navigation context. The monitoring view was used for observation only; mission outcomes and metrics were derived from structured audit logs.

7. Results

Experiments were conducted across three controlled physical-robot sessions on two custom-built differential-drive robots: Xplorer-B, using a single-RPi5 architecture, and Xplorer-C, using a dual-RPi5 architecture. A total of 82 scenario-level decisions were collected during Phase 2: 37 on Xplorer-C, 41 on Xplorer-B, and 4 during concurrent operation (Fig. 7). The seed data used for initial preference promotion was not included in these 82 decisions. After Session A, the memory digest was updated to include the newly promoted S3new preference and then frozen for Sessions B and C (MD5: 97241265, 6 M3 preferences).

7.1. Learning cycle: from VLM reasoning to deterministic resolution

The central experiment demonstrates how the system learns from VLM interactions and converts that knowledge into deterministic behavior. The instruction “Take me somewhere I can sit and relax” was selected because it

requires semantic reasoning about affordances (seating, comfort) that cannot be resolved through direct name matching, object identifiers, or class labels - it escalates to L3b on every encounter.

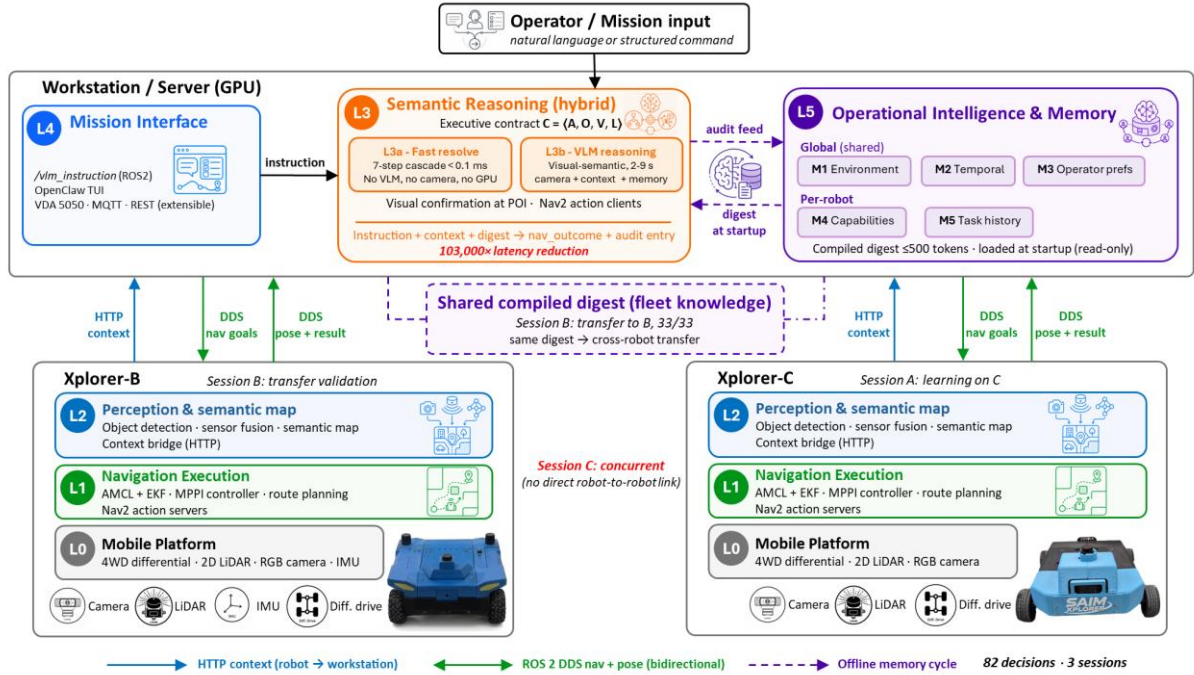


Fig. 7. Validated Xplorer multi-robot deployment. L0–L2 run on each robot's Raspberry Pi 5 (CPU-only); L3 and L5 run on a shared GPU workstation; L4 provides the operator/fleet interface. Blue arrows denote HTTP context requests from each robot's perception bridge; green arrows denote ROS 2 DDS navigation actions and pose feedback. Dashed purple arrows denote the offline memory cycle. Both robots share the same compiled memory digest (≤ 500 tokens) loaded at startup, enabling cross-robot knowledge transfer without direct robot-to-robot communication. *Italic annotations indicate key experimental results: 82 scenario-level decisions across three sessions demonstrated 100% semantic transfer accuracy (33/33) with a measured latency reduction of $103,000\times$ from VLM reasoning to deterministic resolution.*

VLM reasoning phase (Xplorer-C). Across 7 L3b decisions, the VLM resolved the instruction to node 9 (radiator_main, which has attributes *seating: true* and *use: resting spot*) in 6 cases and to node 15 (lab_chair) in 1 case, yielding a consistency of $6/7 = 0.86$. VLM inference times ranged from 2,566 ms to 8,935 ms (mean: $6,733 \pm 2,148$ ms) according with Table 10. A Shapiro–Wilk test did not reject normality ($W = 0.873$, $p = 0.198$), though the sample size limits statistical power.

The single inconsistent resolution (decision 5, node 15) illustrates both VLM non-determinism and the robustness of the promotion mechanism: the consistency threshold of 0.80 is specifically designed to tolerate occasional deviations while requiring a clear majority pattern. At 0.86, the preference was promoted (Fig. 8).

Table 10. Learning cycle: per-decision VLM inference for the instruction “Take me somewhere I can sit and relax” on Xplorer-C. The VLM resolved to the correct node (radiator_main, node 9) in 6 of 7 decisions. The single deviation (decision 5, node 15) did not prevent promotion (consistency $0.86 \geq \text{threshold } 0.80$).

Decision	Target node	Correct	VLM time (ms)	Nav outcome
1	9 (radiator_main)	✓	8,552	mission_complete
2	9 (radiator_main)	✓	8,203	mission_complete
3	9 (radiator_main)	✓	8,935	missed (nav)
4	9 (radiator_main)	✓	2,566	mission_complete
5	15 (lab_chair)	✗	7,921	mission_complete
6	9 (radiator_main)	✓	5,317	mission_complete
7	9 (radiator_main)	✓	5,634	mission_complete
Summary		6/7 (86%)	6,733 $\pm$ 2,148	

Promotion. Following the VLM phase, the memory extractor promoted this instruction to M3. Across all audit logs (including one pre-experimental session), the instruction accumulated a total frequency of 8, with consistency $0.86 \geq 0.80$ and L3b resolution count ≥ 1 - meeting all three promotion criteria. The compiled digest was updated to include this preference alongside three previously promoted preferences from the seed phase (restroom $\rightarrow$ node 0, fire $\rightarrow$ node 8, fresh air $\rightarrow$ node 14).

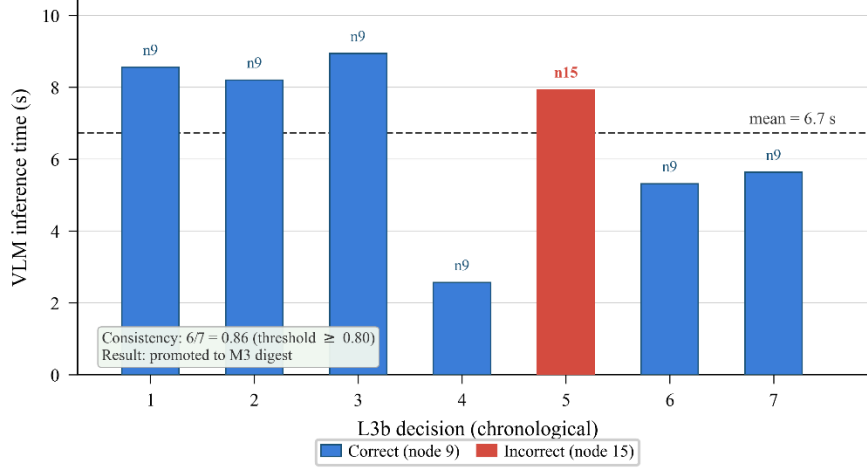


Fig. 8. Learning cycle - VLM inference times on Xplorer-C. Learning cycle: L3b VLM inference times for S3new on Xplorer-C. 7 decisions with correct/incorrect nodes and M3 promotion threshold

Note on implementation refinement. During Session A, an initial set of S3new runs produced inconsistent VLM resolutions due to YOLO false positives (transient chair detections) flooding the VLM context. This was identified as a perception-layer (L2) deficiency - not a reasoning-layer (L3) limitation - and was corrected by introducing a YOLO filtering pipeline (v4.8) that suppresses dynamic objects when static counterparts exist and deduplicates spatially proximate detections. All S3new results reported above use the corrected implementation. The pre-correction data is included in the public dataset for transparency. This episode demonstrates the layered architecture’s diagnostic value: the deficiency was isolated to L2 and corrected without modifying L3, L5, or the promotion logic.

7.2. Cross-robot memory transfer

Session B validated the primary paper claim: preferences learned on Xplorer-C transfer to Xplorer-B without retraining, code modification, or per-robot configuration. Xplorer-B had never received any of the tested instructions prior to this session. The digest used was the same one produced after Session A, containing 6 M3 preferences.

Semantic transfer accuracy. All 33 M3 preference resolutions on Xplorer-B produced the correct target node (Table 11 and Fig. 9). The semantic transfer rate is $33/33 = 100\%$, with a Clopper–Pearson 95% confidence interval of $[0.894, 1.000]$. The result includes both the seed-promoted preferences (S1: 14 runs, S2: 10 runs, fresh air: 6 runs) and the newly promoted S3new (3 runs), confirming that preferences promoted during controlled experiments transfer with the same reliability as those promoted during the seed phase.

Table 11. Cross-robot transfer results on Xplorer-B. All M3 preference resolutions produced the correct target node. Navigation failures (5/33) are attributable to AMCL localization on the single-RPi5 architecture (mean XY error on missed runs: 3.77 m vs 0.30 m on successful runs).

Category	Instruction	N	Resolution method	Target node	Semantic accuracy	Resolve (ms)	Nav success
Seed-promoted	“...short break for personal needs”	14	L3a_m3_preference	0 (toilet_m)	14/14	0.063 ± 0.008	10/14 (71%)
Seed-promoted	“...washroom got on fire...”	10	L3a_m3_preference	8 (cb202)	10/10	0.070 ± 0.010	10/10 (100%)
Seed-promoted	“...too hot...fresh air”	6	L3a_m3_preference	14 (plant_5)	6/6	0.066 ± 0.010	5/6 (83%)
Learning cycle	“...sit and relax”	3	L3a_m3_preference	9 (radiator)	3/3	0.060 ± 0.005	3/3 (100%)
Deterministic	“go to lab_cb204”	5	L3a_deterministic	5 (lab_cb204)	5/5	0.101 ± 0.041	5/5 (100%)
Deterministic	“Take me to the closest plant”	3	L3a_deterministic	19 (plant_3)	3/3	0.400 ± 0.054	3/3 (100%)
All M3		33			33/33 (100%)	0.065 ± 0.009	28/33 (85%)
All deterministic		8			8/8 (100%)	0.213 ± 0.124	8/8 (100%)
Total		41			41/41 (100%)		36/41 (88%)

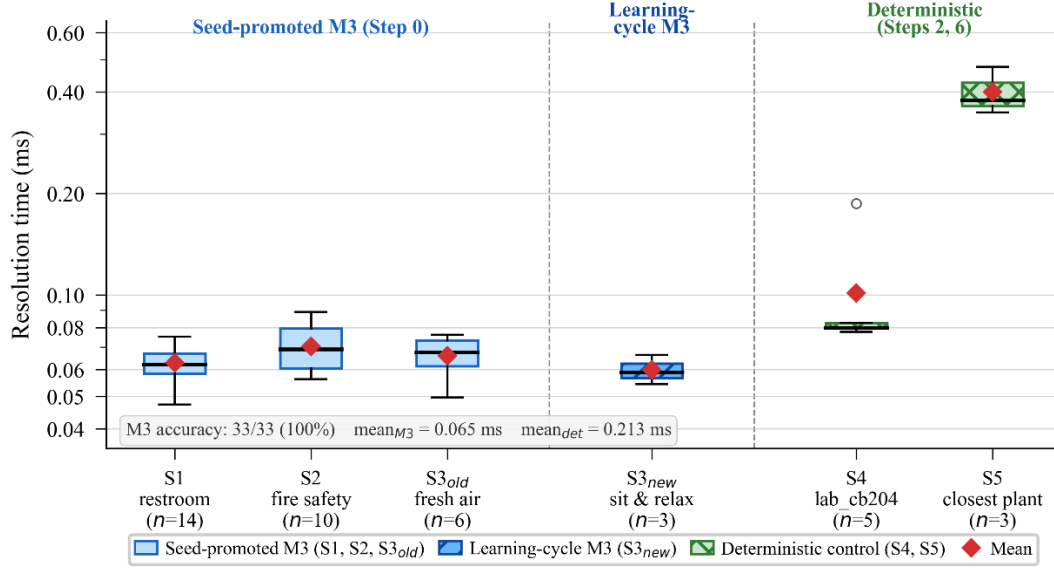


Fig. 9. Transfer verification - L3a resolve times per category on Xplorer-B.

Latency reduction (Fig. 10). The M3 preference resolution on Xplorer-B operated at a mean of 0.065 ms (range: 0.047–0.089 ms), compared to the L3b VLM baseline of 6,733 ms measured during the S3new learning cycle on Xplorer-C. This represents a latency reduction of approximately 103,000 $\times$. A Mann–Whitney U test confirmed this difference is highly significant ($U = 231$, $p = 2.12 \times 10^{-5}$), with a very large effect size (Cohen’s $d = 7.30$). The M3 resolution operates without invoking the VLM, without acquiring a camera image, and without GPU computation. On Xplorer-B, each M3 resolution consumed only Jaccard token matching on the RPi5 CPU - the same operation that would execute on any platform with a text processor, confirming the platform-agnostic nature of the transfer mechanism.

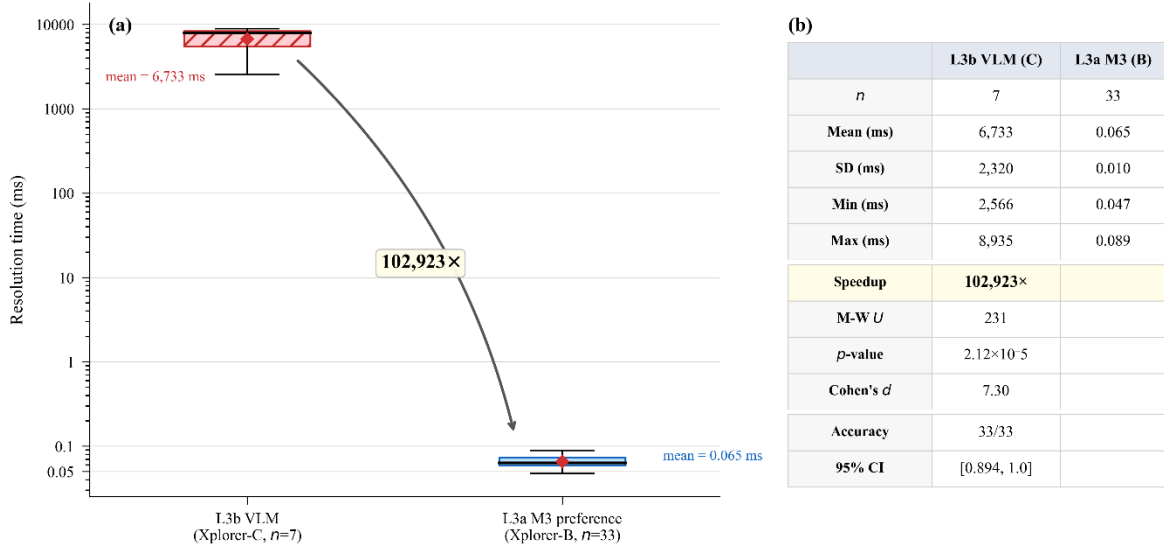


Fig. 10. Latency comparison: L3b VLM (Xplorer-C) vs L3a M3 (Xplorer-B). (a) log-scale boxplot, (b) structured stats table.

Semantic accuracy vs navigation success (Fig. 11). A critical distinction in the results is between semantic resolution accuracy ($41/41 = 100\%$) and navigation completion rate ($36/41 = 88\%$). All 5 navigation failures occurred on M3-resolved instructions (S1: 4 misses, fresh air: 1 miss), while deterministic scenarios achieved 8/8 (100%). However, a Fisher’s exact test found no significant association between resolution method and navigation success ($p = 0.563$), indicating that the failures are attributable to AMCL localization precision on the single-RPi5 architecture rather than to the resolution method. This interpretation is supported by the XY error data: missed runs exhibited a mean error of 3.77 m (AMCL drift beyond goal tolerance), while successful runs averaged 0.30 m. This separation validates the SAS layered architecture: L3 correctness is independent of L1 performance.

Visual confirmation. Post-arrival visual confirmation using VLM landmark matching was executed on scenarios with *signature_type* = *landmark*. A notable observation occurred on S2 (fire): the fire hydrant at node 8 is wall-mounted at approximately 1.5 m height. At the navigation goal position (~0.3 m from the wall), the robot’s forward-facing camera (mounted at 16 cm above ground level) cannot capture the hydrant within its vertical field of view. A portable fire extinguisher placed at floor level near the hydrant serves as an auxiliary visual cue; however, the VLM correctly distinguished it as a fire extinguisher rather than a fire hydrant, reporting *confirmed* = *false* on all 10 runs (XY error 0.29 ± 0.03 m, confirming correct physical arrival). This result reflects a known limitation of low-mounted monocular cameras for confirming wall-mounted objects at close range - a constraint inherent to the 2D sensor operating envelope documented in prior work [7]. The semantic resolution itself was correct in all cases (10/10 to node 8). This observation motivates two planned improvements: a tilt mechanism for the confirmation camera, and depth camera integration (e.g., RealSense D435i) providing a wider vertical field of view.

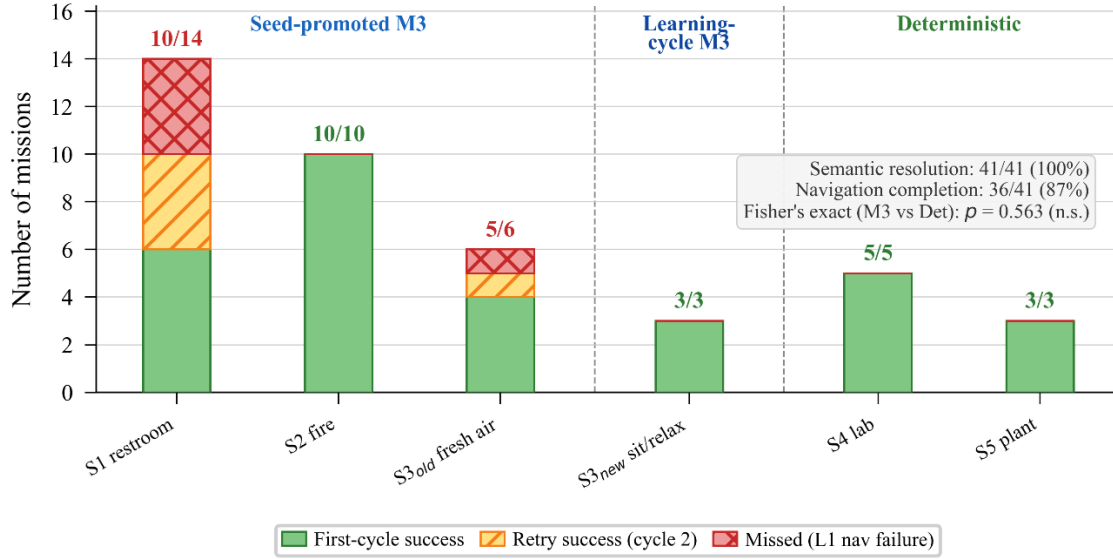


Fig. 11. Navigation outcome distribution on Xplorer-B (Session B). Semantic accuracy (100%) vs navigation completion (88%).

7.3. Deterministic consistency

The deterministic control scenarios verified that the L3a resolver produces identical results on both robots when operating on the same navigation graph and static POI set:

- S4 (“go to lab_cb204”): resolved to node 5 via L3a step 2 (node name match) on both Xplorer-B (5/5) and Xplorer-C (3/3). Mean resolve time: 0.101 ms (B) - slightly higher than M3 resolve due to the cascade traversing Steps 1–2 before matching.
- S5 (“Take me to the closest plant”): resolved to node 19 (plant 3) via L3a step 6 (proximity + class match) on both robots. Mean resolve time: 0.400 ms (B) - the highest among all L3a scenarios, consistent with the additional computational cost of proximity calculation across multiple plant POIs.

The deterministic resolve times were significantly higher than M3 preference resolve times (Mann–Whitney $U = 254$, $p < 0.001$), which is architecturally expected: M3 matching (Step 0) short-circuits the cascade before Steps 1–6 are evaluated, while deterministic resolution must traverse the full cascade until a matching step is found.

7.4. Concurrent operation

Session C demonstrated that two navigator instances can operate simultaneously with a shared memory digest, producing correct and distinct results. Two pairs of missions were executed with robots starting from opposite ends of the corridor and navigating to different destinations:

Table 12. Concurrent operation results (Session C). Both navigator instances used the same digest (MD5: 97241265). Robots navigated in opposite directions to avoid physical conflict.

Pair	Xplorer-B	B node	B outcome	Xplorer-C	C node	C outcome
1	S2 (fire)	8 ✓	mission_complete	S1 (restroom)	0 ✓	mission_complete
2	cb203 entrance	22 ✓	mission_complete	lab_cb204	5 ✓	mission_complete

Integrity verification confirmed: both startup events recorded the same digest hash, B audit files contained only *platform_id* = *xplorer-b* entries, C audit files contained only *platform_id* = *xplorer-c* entries, and no DDS cross-talk was observed between the isolated domains.

Both pairs used L3a-resolved instructions (M3 preference and deterministic respectively). Concurrent VLM serialization - where both robots simultaneously escalate to L3b and share the Ollama instance - was not tested and is identified as future work.

7.5. Summary of statistical tests

Table 13. Summary of statistical tests applied to the experimental data.

Test	Variables	Statistic	p-value	Interpretation
Exact binomial (Clopper–Pearson)	M3 transfer accuracy: 33/33	-	-	100%, 95% CI [0.894, 1.000]
Mann–Whitney U	L3b time (C, n=7) vs L3a time (B, n=33)	U = 231	2.12×10^{-5}	Highly significant latency reduction
Cohen’s d	L3b vs L3a timing	d = 7.30	-	Very large effect size
Shapiro–Wilk	M3 resolve times (B, n=33)	W = 0.977	0.683	Normally distributed
Shapiro–Wilk	Deterministic resolve (B, n=8)	W = 0.809	0.036	Non-normal (step 6 proximity cost)
Shapiro–Wilk	S3new VLM times (C, n=7)	W = 0.873	0.198	Not rejected (limited power)
Fisher’s exact	M3 vs Det navigation success on B	-	0.563	No significant association
Mann–Whitney U	Deterministic vs M3 resolve time on B	U = 254	< 0.001	M3 (Step 0) significantly faster than Steps 1–6

8. Discussion

8.1. Key findings

The experimental results support the four contributions claimed in this work. Five principal findings are discussed below.

Finding 1: The deterministic resolver handles the majority of instructions without VLM involvement.

Across all 82 scenario decisions, 72 (88%) were resolved by L3a - either through M3 preference matching (Step 0) or through the deterministic cascade (Steps 1–6). Only 10 decisions (12%) required L3b VLM inference, all of which were S3new learning cycle runs on Xplorer-C. On Xplorer-B (the transfer robot), 100% of decisions were resolved by L3a, meaning the VLM was never invoked. This supports the design rationale (Section 4.1): in the tested semantic-navigation scenarios, a large fraction of indoor navigation instructions could be resolved through graph structure, semantic annotations, and learned preference matching without the latency and compute cost of VLM inference.

Finding 2: The learning cycle from VLM to deterministic resolution is complete and measurable. The S3new instruction (“Take me somewhere I can sit and relax”) traversed the full pipeline: 7 L3b VLM decisions on Xplorer-C (mean: 6,733 ms, consistency: 0.86) → memory extraction with promotion (frequency = 8, consistency ≥ 0.80 , L3b count ≥ 1) → deterministic resolution on Xplorer-B via L3a Step 0 (mean: 0.060 ms, 3/3 correct). The measured latency reduction of $103,000\times$ (Mann–Whitney U = 231, $p = 2.12 \times 10^{-5}$, Cohen’s d = 7.30) demonstrates that the adaptation mechanism produces a practically significant performance improvement, not merely a statistically significant one.

Finding 3: Cross-robot memory transfer achieved 100% semantic accuracy in the tested transfer scenarios.

All 33 M3 preference resolutions on Xplorer-B produced the predefined correct target node (100%, Clopper–Pearson 95% CI [0.894, 1.000]). This includes both seed-promoted preferences and the experimentally promoted S3new preference. The transfer required no retraining and no per-robot rule engineering; the operational requirement was that both robots loaded the same compiled digest and shared compatible graph and POI identifiers.

Finding 4: Semantic resolution accuracy and navigation success measure different capabilities. A critical distinction emerged from the data: semantic resolution was 41/41 (100%) on Xplorer-B, while navigation completion was 36/41 (88%). The five missed runs showed substantially larger XY error than successful runs (3.77 m vs. 0.30 m), and Fisher’s exact test found no significant association between resolution method and navigation completion ($p = 0.563$). Together, these results support the interpretation that the missed runs were primarily related to localization/navigation-layer performance rather than to incorrect L3 target resolution. This separation supports the SAS layered interpretation: L3 target selection can be evaluated independently from L1 navigation execution.

Finding 5: Concurrent operation is feasible with shared digest and DDS isolation. Two navigator instances operating simultaneously—each communicating with a different robot on a separate DDS domain—produced correct results with distinct audit logs, identical digest hashes, and no observed DDS cross-talk. This test was limited to four L3a-resolved decisions and did not evaluate simultaneous L3b escalation or parallel VLM inference.

Therefore, the result should be interpreted as a feasibility demonstration of the multi-robot deployment architecture, not as a full fleet-scale validation.

8.2. Comparison with related work

Table 14 compares the proposed framework with selected representative systems from the related-work categories discussed in Section 2: dual-process VLM navigation, semantic-memory navigation, LLM–ROS integration frameworks, and the authors’ prior semantic route-planning work. The comparison is qualitative and should be interpreted in terms of architectural capabilities and validation scope, not as a benchmark-level performance ranking.

Table 14. Comparison with related systems. Entries marked “-” indicate capabilities not addressed or not documented. “Partial” indicates limited coverage.

Dimension	IROS [6]	Hydra-Nav [19]	CausalNav [22]	ROSClaw [26]	ROS-LLM [27]	Sensors [7]	This work
Fast-resolve mechanism	Trained RL classifier	Adaptive CoT in VLM	-	-	-	N/A (L1–L2)	7-step parametric cascade
Fast-path rate	53.6%	Not reported	-	-	-	-	88%
VLM/LLM role	With bypass	Unified (single VLM)	Implicit (RAG)	Generic tool-use	Task programming	-	Selective (L3b only when L3a fails)
Memory categories	-	-	Causal graph (1)	Generic plugin	-	-	M1–M5 (5 with scope taxonomy)
Cross-session learning	-	-	Yes (graph update)	Plugin-dependent	-	-	Yes (M3 promotion)
Cross-robot transfer	-	-	-	-	-	-	Yes (33/33 = 100%)
Physical validation	5 buildings	Simulation	Sim + real lab	3 platforms, lab	Lab tasks	3 robots, 115 legs	2 robots, 82 decisions, 3 sessions
Training required	RL classifier	RL fine-tuning	Model-specific	None	Imitation learning	None	None

Comparison with IROS dual-process navigation [6]. The closest related work applies dual-process theory to VLM-based navigation by training an RL classifier to route decisions to fast or slow pathways. IROS reports a 53.6% System 1 rate and a 66% latency reduction across five real-world buildings. The present work pursues the same high-level goal-reducing unnecessary VLM invocation-but uses a different routing mechanism: a seven-step parametric cascade exploiting graph structure, semantic annotations, and promoted preferences. In the tested SAS scenarios, this produced an 88% fast-path rate. Because the experimental protocols, environments, and instruction sets differ, this value should not be interpreted as a direct benchmark-level superiority claim over IROS. The more important distinction is architectural: the SAS fast path requires no learned classifier, no task-specific training data, and supports cross-session and cross-robot preference reuse.

Comparison with Hydra-Nav [19]. Hydra-Nav unifies dual-process reasoning within a single VLM, using RL-trained adaptive switching between direct execution and Chain-of-Thought reasoning for object navigation. The present work differs in that the fast path is entirely external to the VLM - a parametric cascade that does not invoke any language model - enabling sub-millisecond resolution and eliminating GPU dependency for routine instructions. Also, Hydra-Nav is evaluated on object-navigation benchmarks, whereas the present work focuses on physical ROS 2/Nav2 deployment, audit-log-based evaluation, and cross-robot reuse of learned semantic preferences.

Comparison with CausalNav [22]. CausalNav proposes a dynamic causal semantic graph for outdoor navigation with bidirectional LLM–RAG reasoning. CausalNav focuses on maintaining and updating a dynamic causal semantic graph for navigation with LLM–RAG reasoning. The present work addresses a complementary memory problem: separating operational memory into five categories (M1–M5) with explicit scope differentiation, so that environment knowledge, operator preferences, platform capabilities, and task history can be handled differently. In particular, the M3 mechanism promotes repeated VLM-mediated operator preferences into deterministic rules and makes them reusable across robots sharing the same semantic map.

Comparison with ROSClaw [26]. ROSClaw provides a model-agnostic executive layer for agentic robot control through LLM tool-calling and formalizes the contract $C = \langle A, O, V, L \rangle$. The present work adopts this contract perspective and specializes it for semantic navigation: the affordance manifest contains navigation graph nodes and POI attributes, the validator enforces navigation-specific constraints, and the logger records decision-level

metrics with platform attribution. ROSClaw is intentionally general-purpose, whereas the SAS implementation is domain-specific: it resolves natural-language intent to graph targets, executes Nav2 navigation, performs post-arrival confirmation, and supports memory-driven cross-robot transfer.

Comparison with ROS-LLM [27]. Published in Nature Machine Intelligence in 2026, ROS-LLM provides a comprehensive framework for embodied AI with task feedback, imitation learning, and structured reasoning via behavior trees. The key distinction is the level of abstraction: ROS-LLM operates at the task programming level (generating ROS action sequences from natural language), while the present work operates at the semantic navigation level (interpreting intent, resolving targets on a navigation graph, confirming arrival). ROS-LLM supports feedback-driven behavior generation and imitation learning for extending skills. The present work is complementary: it does not target general task programming, but focuses specifically on semantic navigation, where the central problem is resolving operator intent to a graph target, validating the action, executing Nav2 navigation, and learning reusable preferences from operational data.

Comparison with the authors’ prior Sensors work [7]. The Sensors paper validated L1–L2 of the SAS framework: perception (ASF pipeline), persistent semantic map, and penalty-weighted route planning across 115 navigation legs on 3 robots with a 97% success rate. The present work adds L3 (hybrid semantic reasoning) and L5 (adaptive memory with cross-robot transfer), completing the SAS validation from perception through reasoning to memory.

8.3. Limitations

The following limitations should be considered when interpreting the results.

Single environment. All experiments were conducted in a single indoor corridor (FIIR second floor) with a fixed 24-node navigation graph. Generalization to larger spaces, different building types, or outdoor environments has not been validated. The navigation graph size (24 nodes, 18 POIs) is sufficient to demonstrate the architecture but does not stress-test scalability. Future work will extend validation to a multi-room layout spanning the full FIIR floor (~50 nodes). The single-environment design was intentional for this first validation, because cross-robot memory transfer requires a controlled shared map, graph, and POI set. However, it limits external validity and does not yet demonstrate robustness across buildings.

Lexical matching for M3 preferences. The M3 promotion and runtime matching use Jaccard similarity on token sets, not semantic embeddings. Paraphrased instructions with low lexical overlap (e.g., “I need to use the bathroom” vs “go to a place to take a short break for personal needs”) may not match, fragmenting into separate preference groups. The Jaccard threshold of 0.6 provides tolerance for minor wording variations but would fail on fundamentally different phrasings of the same intent. Replacing Jaccard with a lightweight sentence embedding model is planned for future work.

VLM non-determinism. The S3new learning cycle showed 86% consistency (6/7 correct). While this exceeds the promotion threshold (0.80), it demonstrates that VLM reasoning is inherently variable - decision 5 resolved to a plausible but incorrect node (lab_chair instead of radiator). The promotion mechanism is designed to tolerate this variability, but lower-consistency instructions (close to the 0.80 threshold) may oscillate between promoted and not-promoted states across extraction runs.

Camera field of view for visual confirmation. The forward-facing camera (mounted at 16 cm height) cannot capture wall-mounted objects at close range. The fire hydrant at node 8 (wall-mounted at ~1.5 m height) was correctly identified as absent by the VLM on all 10 confirmation attempts - the system correctly reported *confirmed = false* because the hydrant was outside the camera’s vertical field of view at the arrival distance (~0.3 m from wall). This result illustrates both the usefulness and the limitation of the confirmation mechanism: the VLM did not falsely confirm the expected wall-mounted object when it was outside the camera field of view, but the current sensor geometry prevented confirmation of a semantically correct arrival.

VLM model scope. All experiments used Qwen 3.5:4b (4 billion parameters). While the executive contract (Section 4.5) provides model agnosticism, performance has not been experimentally validated with larger models (7B, 14B) or different model families (Gemma, LLaVA). The model-agnostic claim is architectural, not empirically validated across models. VLM benchmarking under the same contract is planned as immediate future work.

Concurrent VLM serialization not tested. Because L3b was invoked in only 12% of the tested decisions, simultaneous L3b escalation was not observed in the controlled sessions. However, in bursty or highly ambiguous workloads, multiple robots could request VLM inference at the same time, producing queueing latency in the current single-Ollama deployment.

8.4. Threats to validity

Internal validity. The same operator conducted all experiments, using the same instruction phrasings across sessions. M3 preferences may not generalize to operators with different linguistic habits. The v4.8 implementation refinement during Session A (YOLO filtering fix) introduces a confound: pre- and post-fix data

were collected under different code versions. This is mitigated by using only post-fix data in the reported results and documenting the pre-fix data separately.

External validity. All experiments were conducted in a single indoor environment with a fixed set of objects. The 18 POIs represent a moderately rich but not exhaustive indoor annotation. Environments with substantially more objects, higher ambiguity (multiple similar objects), or different spatial layouts may challenge the L3a cascade differently. The proximity resolution (Step 6) is geometry-dependent: in a symmetric layout, “nearest plant” may be ambiguous.

Construct validity. Semantic resolution accuracy (100%) and navigation completion (88%) measure different constructs. A system that always resolves to the correct node but never reaches it would score 100%/0%. The reporting separates these measures explicitly, and the Fisher’s exact test ($p = 0.563$) confirms that navigation failures are not associated with the resolution method. The visual confirmation mechanism provides an additional construct - arrival verification - but its accuracy is bounded by the camera’s physical capabilities (Section 8.3).

9. Conclusions and Future Work

9.1. Conclusions

This paper presented the Semantic Autonomy Stack (SAS), a six-layer reference framework for semantically autonomous indoor mobile robot navigation, and validated a complete instance featuring hybrid deterministic-VLM reasoning and cross-robot adaptive memory on two physical ROS 2/Nav2 robots. The mobile platforms used off-the-shelf Raspberry Pi 5 hardware for onboard navigation and perception, while VLM inference was executed on a shared consumer GPU workstation.

Four contributions were evaluated through 82 scenario-level decisions collected across two custom-built mobile robots and three controlled physical-robot sessions.

C1 - Semantic Autonomy Stack. The SAS framework (L0–L5) provides a structured decomposition of the capabilities required for semantically autonomous indoor navigation. The framework is agnostic to specific sensors, VLM models, and locomotion platforms - each layer prescribes what must be provided, not how it is implemented. The layered design also illustrated its diagnostic value during the experiments: a perception-layer (L2) issue related to YOLO false positives was identified and corrected without modifying the reasoning (L3) or memory (L5) layers.

C2 - Hybrid deterministic-VLM reasoning. The seven-step parametric resolver (L3a) handled 72 of 82 scenario-level decisions (88%) without invoking the Vision-Language Model, acquiring a camera image, or requiring GPU acceleration for the deterministic fast path. When L3a could not resolve an instruction, L3b activated the VLM (Qwen 3.5:4b) with on-demand camera acquisition, structured prompt construction, and visual confirmation post-arrival. The learning cycle from L3b to L3a was demonstrated end-to-end: a new instruction (“Take me somewhere I can sit and relax”) was resolved 7 times by the VLM (mean: 6,733 ms, consistency: 0.86), promoted to M3 memory, and subsequently resolved deterministically in 0.060 ms - a latency reduction of 103,000 $\times$.

C3 - Cross-robot adaptive memory. The five-category memory framework (M1–M5) with explicit scope taxonomy (global environment knowledge, per-operator preferences, per-robot capabilities) enabled cross-session learning and cross-robot transfer. All 33 M3 preference resolutions on Xplorer-B produced the predefined correct target node (100%, Clopper–Pearson 95% CI [0.894, 1.000]). The transfer required no retraining and no per-robot rule engineering; the operational requirement was that both robots shared compatible graph and POI identifiers and loaded the same compact compiled memory digest at startup.

C4 - Physical multi-robot validation. The framework was validated on two heterogeneous robot platforms (single-RPi5 and dual-RPi5 architectures) with off-the-shelf edge hardware and no onboard GPU. Semantic resolution accuracy was 100% across all scenarios. Navigation completion was 88%. The observed missed runs were associated with larger XY errors and support the interpretation that the failures were primarily related to localization/navigation-layer performance rather than to incorrect L3 target resolution.

9.2. Future work

Six directions are prioritized for future development, ordered by expected impact and feasibility.

VLM benchmarking. The current implementation uses Qwen 3.5:4b exclusively. While the executive contract (Section 4.5) guarantees model agnosticism at the interface level, empirical validation across model families (Qwen 2.5-VL-7B, Gemma 4 E4B) and sizes (4B, 7B, 14B) is needed to characterize accuracy–latency tradeoffs and confirm that the promotion mechanism operates consistently across models. The benchmark protocol will use identical prompts, graph, and policy - varying only the model identifier.

Semantic embeddings for M3 matching. The current M3 preference matching uses Jaccard similarity on token sets, which requires high lexical overlap between the stored and incoming instructions. Replacing the Jaccard matcher with a lightweight sentence embedding model (e.g., all-MiniLM-L6-v2, 22M parameters, <10 ms inference on CPU) would enable matching of semantically equivalent but lexically different phrasings (“I need

to use the bathroom” → matches “go to a place to take a short break for personal needs”). The M3 promotion criteria and digest structure would remain unchanged.

Platform portability. The SAS framework is designed to be platform-agnostic above L0, but this claim must be tested on platforms that differ more substantially from the Xplorer differential-drive robots. A planned validation target is the Unitree Go2 quadruped robot equipped with 3D LiDAR and depth sensing. This experiment will test whether the L3–L5 reasoning and memory layers can be reused with minimal modification once the L1 navigation interface and L2 semantic perception outputs are adapted to the new platform.

Edge GPU deployment. The current architecture distributes L3 (reasoning) and L5 (memory) to an operator workstation with a consumer GPU. Migrating L3 to an on-robot NVIDIA Jetson Orin would make the navigator self-contained, eliminating the workstation dependency. The HTTP bridge would become localhost calls; DDS actions would remain identical. The memory digest would transition from a shared file to a network-served artifact, with robots pulling updates from a central memory server.

Shared memory server. For fleet deployments beyond two robots, the manual memory extraction model will be replaced by a REST API memory server. Robots push audit logs asynchronously, the server runs the extractor periodically, and robots pull updated digests at startup with local cache fallback. The M1–M5 scope taxonomy was designed for this transition: global categories (M1–M3) are served to all robots, per-robot categories (M4, M5) are served only to the requesting platform. Integration with industrial fleet management protocols (VDA 5050) at L4 is planned to enable interoperability with commercial AMR fleet managers.

Larger navigation graphs and environments. The current validation uses a 24-node graph in a single corridor. Extending to a full-floor graph (~50 nodes spanning multiple connected corridors, ~200 m traversable path) will test scalability of the L3a cascade, the Route Server planning time under increased graph complexity, and the M3 preference disambiguation when more POIs compete for the same instruction patterns. Multi-floor navigation with elevator integration is a longer-term objective.

Overall, the results show that semantically autonomous indoor navigation does not require VLM invocation for every operator instruction. By combining deterministic graph- and memory-based resolution with selective VLM reasoning and scoped semantic memory, the proposed SAS instance provides a practical path toward adaptive, auditable, and transferable semantic navigation on physical ROS 2 mobile robots.

Data Availability Statement

The experimental dataset — including structured audit logs (JSONL), memory files (M1–M5), compiled digest, semantic POI configuration, and analysis scripts for reproducing all figures and statistical tests — is publicly available at: <https://github.com/bogdan-abaza/nav2-sas-vlm-memory>. Source code for the navigator, executive contract, and memory extractor is available from the corresponding author upon reasonable request.

CRedit Author Contributions

Bogdan Felician Abaza: Conceptualization, Methodology, Software (SAS framework, semantic memory design, test framework), Formal analysis, Writing – original draft, Writing – review and editing, Validation, Visualization, Supervision, Project administration.

Andrei-Alexandru Staicu: Software (VLM navigator, context bridge, executive contract, YOLO integration, network optimization), Investigation, Data curation, Writing – review and editing, Validation, Visualization.

Cristian Vasile Doicin: Resources, Funding acquisition, Formal analysis, Writing - review & editing, Validation, Visualization, Supervision

Funding

This research was supported by the National University of Science and Technology POLITEHNICA Bucharest (UNSTPB).

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2," *Robotics and Autonomous Systems*, vol. 168, p. 104525, 2023. doi: 10.1016/j.robot.2023.104525.
- [2] S. Macenski et al., "Model Predictive Path Integral Controller for Nav2," presented at ROSCon 2023. Available: <https://navigation.ros.org/configuration/packages/configuring-mppic.html>.
- [3] C. Zhang, Q. Xu, Y. Yu, G. Zhou, K. Zeng, F. Chang, and K. Ding, "A survey on potentials, pathways and challenges of large language models in new-generation intelligent manufacturing," *Robotics and Computer-Integrated Manufacturing*, vol. 92, Article 102883, 2025. doi: 10.1016/j.rcim.2024.102883.
- [4] R. Alqobali, M. Alshmrani, R. Alnasser, A. Rashidi, and T. Alhmiedat, "A Survey on Robot Semantic Navigation Systems for Indoor Environments," *Applied Sciences*, vol. 14, no. 1, p. 89, 2024. doi: 10.3390/app14010089.
- [5] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.

- [6] J. Lee, H. Shin, and J. Ko, "IROS: A Dual-Process Architecture for Real-Time VLM-Based Indoor Navigation," arXiv preprint arXiv:2601.21506, 2026.
- [7] B. F. Abaza, A.-A. Staicu, and C. V. Doicin, "Lightweight Semantic-Aware Route Planning with Monocular Camera-2D LiDAR Fusion for Indoor Mobile Robots," *Sensors*, vol. 26, no. 7, p. 2232, 2026. doi: 10.3390/s26072232.
- [8] R. Shao, W. Li, L. Zhang, R. Zhang, Z. Liu, R. Chen, and L. Nie, "Large VLM-based Vision-Language-Action Models for Robotic Manipulation: A Survey," arXiv preprint arXiv:2508.13073, 2025.
- [9] D. Zhang, J. Sun, C. Hu, X. Wu, Z. Yuan, R. Zhou, F. Shen, and Q. Zhou, "Pure Vision Language Action (VLA) Models: A Comprehensive Survey," arXiv preprint arXiv:2509.19012, 2025.
- [10] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar, "Actions as Language: Fine-Tuning VLMs into VLAs Without Catastrophic Forgetting," in *Proc. ICLR*, 2025. arXiv:2509.22195.
- [11] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation," arXiv preprint arXiv:2508.19236, 2025.
- [12] B. Chandaka, G. X. Wang, H. Chen, H. Che, A. J. Zhai, and S. Wang, "Human-like Navigation in a World Built for Humans," in *Proc. CoRL*, Seoul, Korea, 2025. arXiv:2509.21189.
- [13] D. Song, J. Liang, A. Payandeh, A. H. Raj, X. Xiao, and D. Manocha, "VLM-Social-Nav: Socially Aware Robot Navigation through Scoring using Vision-Language Models," arXiv preprint arXiv:2404.00210, 2024.
- [14] T. Wang et al., "VLM-Based Human-Guided Mobile Robot Navigation in an Unstructured Environment for Human-Centric Smart Manufacturing," *Engineering*, 2025. doi: 10.1016/j.eng.2025.04.028.
- [15] B. Yu, Q. Yuan, K. Li, H. Kasaei, and M. Cao, "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv preprint arXiv:2310.07937, 2023, revised 2025.
- [16] T. Windecker, M. Patel, M. Reuss, R. Schwarzkopf, C. Cadena, R. Lioutikov, M. Hutter, and J. Frey, "NaviTrace: Evaluating Embodied Navigation of Vision-Language Models," arXiv preprint arXiv:2510.26909, 2025.
- [17] G. Monaci, R. S. Rezende, R. Deffayet, G. Csurka, G. Bono, H. Déjean, S. Clinchant, and C. Wolf, "RANA: Retrieval-Augmented Navigation," arXiv preprint arXiv:2504.03524, 2025.
- [18] Y. Huang, L. Liu, S. Lei, Y. Ma, H. Su, J. Mei, P. Zhao, Y. Gu, Y. Liu, and J. Lv, "CogDDN: A Cognitive Demand-Driven Navigation with Decision Optimization and Dual-Process Thinking," in *Proc. 33rd ACM Int. Conf. Multimedia (MM '25)*, Dublin, Ireland, 2025, pp. 1–10. doi: 10.1145/3746027.3755832.
- [19] Z. Wang, H. Fang, S. Wang, Y. Luo, H. Dong, W. Li, and Y. Gan, "Hydra-Nav: Object Navigation via Adaptive Dual-Process Reasoning," arXiv preprint arXiv:2602.09972, 2026.
- [20] B. Han, J. Kim, and J. Jang, "A Dual Process VLA: Efficient Robotic Manipulation Leveraging VLM," arXiv preprint arXiv:2410.15549, 2024.
- [21] L. Zhang, X. Hao, Q. Xu, Q. Zhang, X. Zhang, P. Wang, J. Zhang, Z. Wang, S. Zhang, and R. Xu, "MapNav: A Novel Memory Representation via Annotated Semantic Maps for VLM-based Vision-and-Language Navigation," in *Proc. ACL (Volume 1: Long Papers)*, Vienna, Austria, 2025, pp. 13032–13056. doi: 10.18653/v1/2025.acl-long.638.
- [22] H. Duan, S. Luo, Z. Deng, Y. Chen, Y. Chiang, Y. Liu, F. Liu, and X. Wang, "CausalNav: A Long-term Embodied Navigation System for Autonomous Mobile Robots in Dynamic Outdoor Scenarios," *IEEE Robotics and Automation Letters*, 2026. arXiv:2601.01872.
- [23] Y. Mao, H. Ye, W. Dong, C. Zhang, and H. Zhang, "Meta-Memory: Retrieving and Integrating Semantic-Spatial Memories for Robot Spatial Reasoning," arXiv preprint arXiv:2509.20754, 2025.
- [24] M. Lin, X. Liang, B. Lin, L. Jingzhi, Z. Jiao, K. Li, Y. Sun, W. Liufu, Y. Ma, Y. Liu, S. Zhao, Y. Zhuang, and X. Liang, "EchoVLA: Synergistic Declarative Memory for VLA-Driven Mobile Manipulation," arXiv preprint arXiv:2511.18112, 2025.
- [25] Y. Hu, S. Liu, Y. Yue, G. Zhang et al., "Memory in the Age of AI Agents," arXiv preprint arXiv:2512.13564, 2025.
- [26] J. Cardenas et al., "ROSClaw: OpenClaw ROS 2 Framework for Agentic Robot Control," arXiv preprint arXiv:2603.26997, 2026.
- [27] A. Mower, S. Wan et al., "A Robot Operating System Framework for Using Large Language Models in Embodied AI," *Nature Machine Intelligence*, 2026. doi: 10.1038/s42256-026-01186-z.
- [28] R. Royce, M. Kaufmann, J. Beckett, S. Moon, K. Carpenter, K. Pak, A. Towler, R. Thakker, and S. Khattak, "Enabling Novel Mission Operations and Interactions with ROSA: The Robot Operating System Agent," arXiv preprint arXiv:2410.06472, 2024.
- [29] B. Rachwał et al., "RAI: A Flexible Agent Framework for Embodied AI," arXiv preprint arXiv:2505.07532, 2025.
- [30] E. K. Raptis, A. Ch. Kapoutsis, and E. B. Kosmatopoulos, "Agentic LLM-based Robotic Systems for Real-World Applications: A Review on Their Agenticness and Ethics," *Frontiers in Robotics and AI*, vol. 12, p. 1605405, 2025. doi: 10.3389/frobt.2025.1605405.

- [31] D. Wu, P. Zheng, Q. Zhao, S. Zhang, J. Qi, J. Hu, G.-N. Zhu, and L. Wang, "Empowering natural human–robot collaboration through multimodal language models and spatial intelligence: Pathways and perspectives," *Robotics and Computer-Integrated Manufacturing*, vol. 97, Article 103064, 2026. doi: 10.1016/j.rcim.2025.103064.
- [32] K. Ding, Q. Mao, Y. Zhang, Y. Zhang, P. Zheng, and L. Wang, "Review and perspectives on multimodal perception, mutual cognition, and embodied execution for human–robot collaboration in Industry 5.0," *Robotics and Computer-Integrated Manufacturing*, vol. 101, p. 103280, 2026. doi: 10.1016/j.rcim.2026.103280.
- [33] VDA 5050: Interface for the Communication between Automated Guided Vehicles (AGV) and a Master Control. Version 2.0.0. VDA Technical Committee. <https://github.com/VDA5050/VDA5050>.
- [34] J. Chen, S. Huang, X. Wang, P. Wang, J. Zhu, Z. Xu, G. Wang, Y. Yan, and L. Wang, "Perception-decision-execution coordination mechanism driven dynamic autonomous collaboration method for human-like collaborative robot based on multimodal large language model," *Robotics and Computer-Integrated Manufacturing*, vol. 98, Article 103167, 2026. doi: 10.1016/j.rcim.2025.103167.